

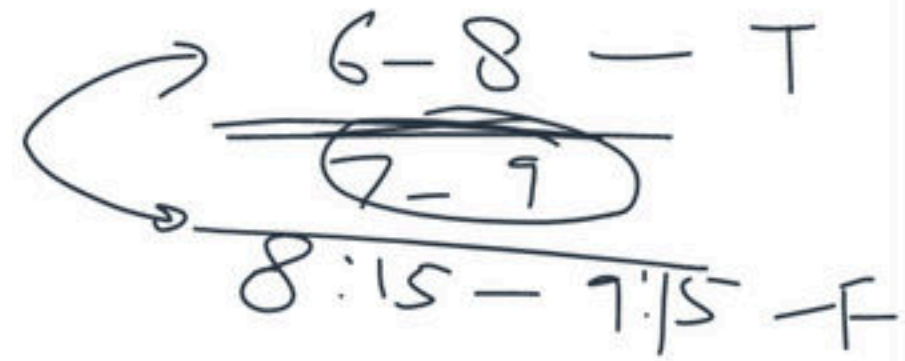


Doubt Clearing Session

Foundation Course on Operating Systems for GATE 2022

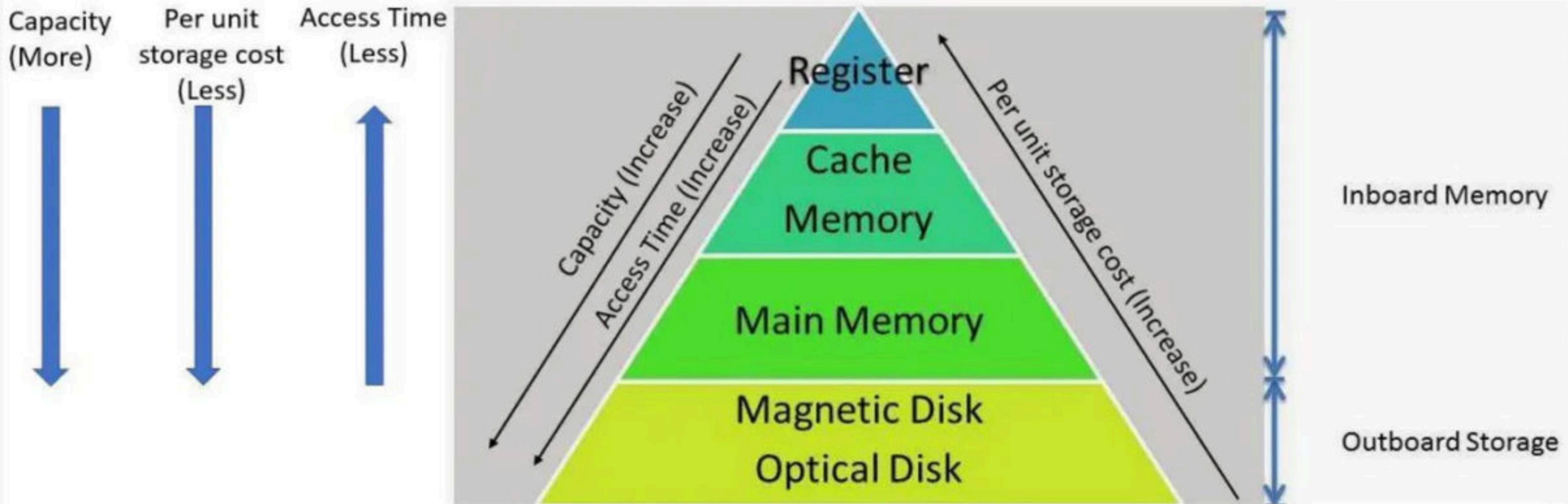
Memory Hierarchy

- Let first understand what we need from a memory
 - Large capacity
 - Less per unit cost
 - Less access time(fast access)



Memory Hierarchy

- The memory hierarchy system consists of all storage devices employed in a computer system from the slow but high-capacity auxiliary memory to a relatively faster main memory, to an even smaller and faster cache memory accessible to the high-speed





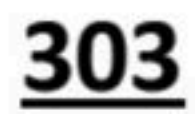
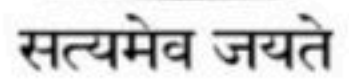
Cycle

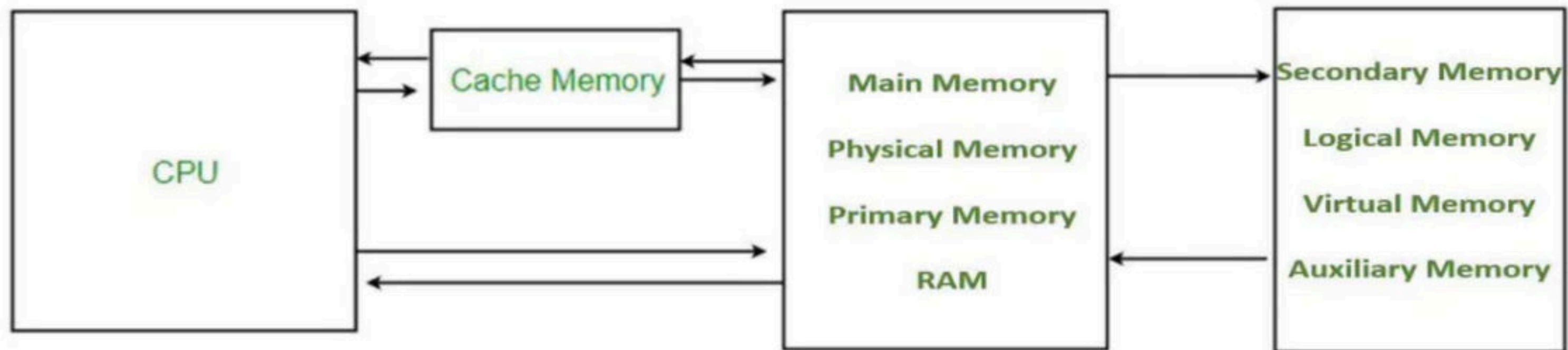
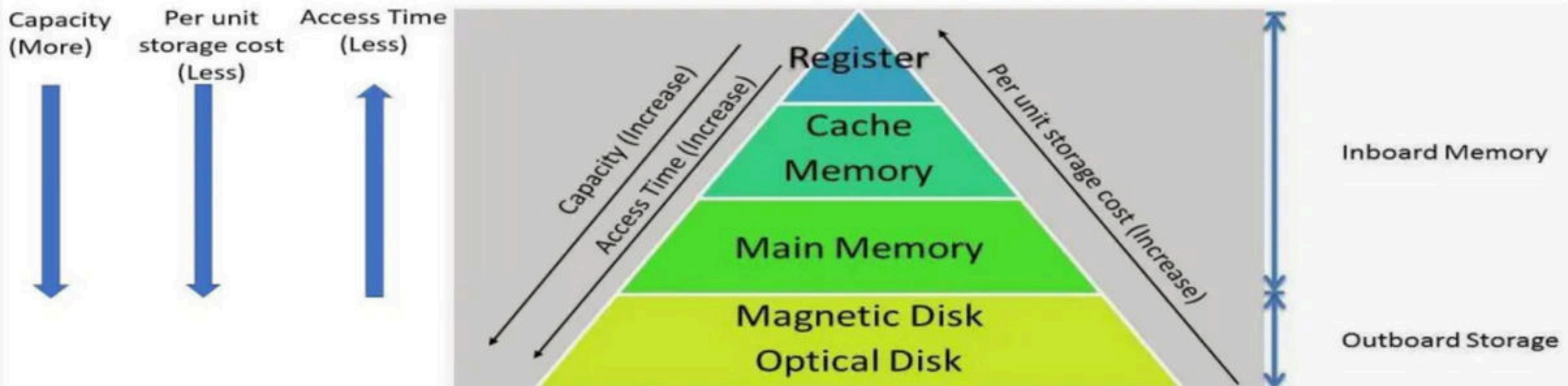


Car



Airbus







Showroom

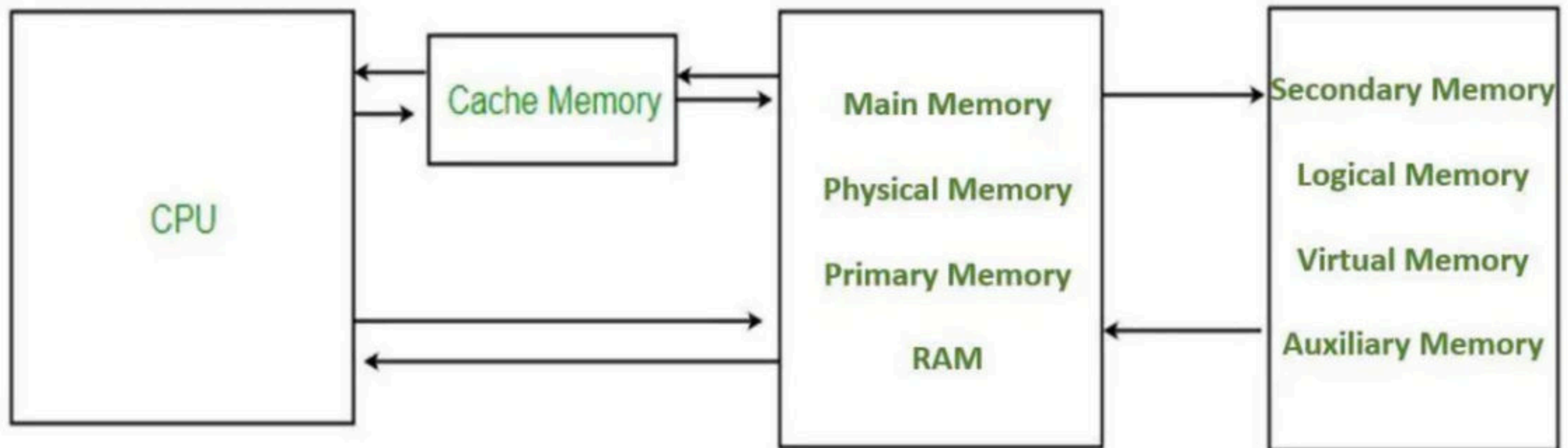


Go down

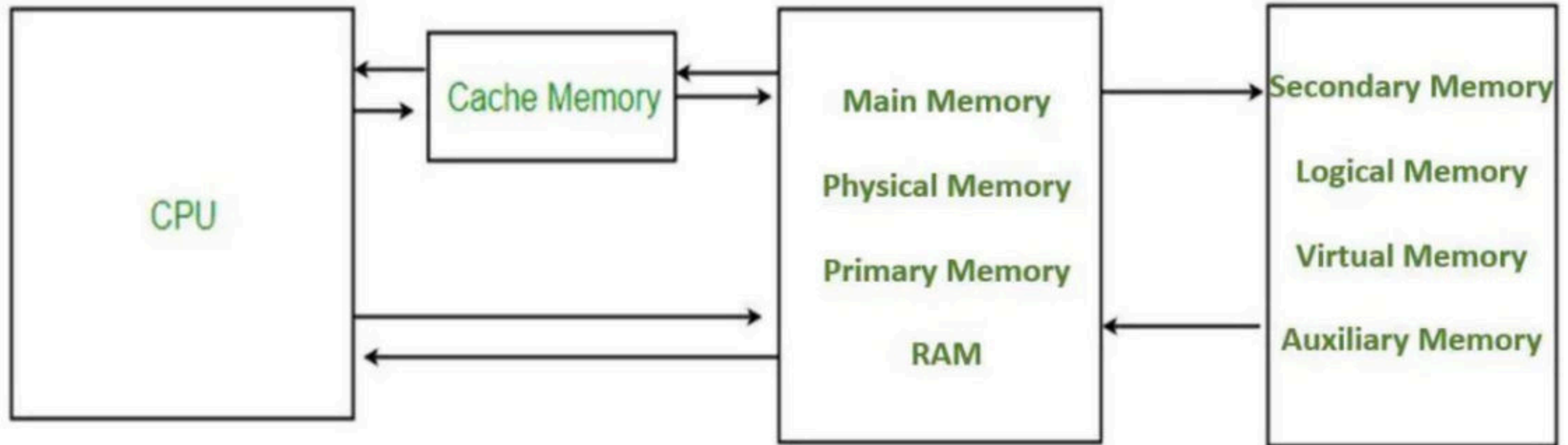


Factory

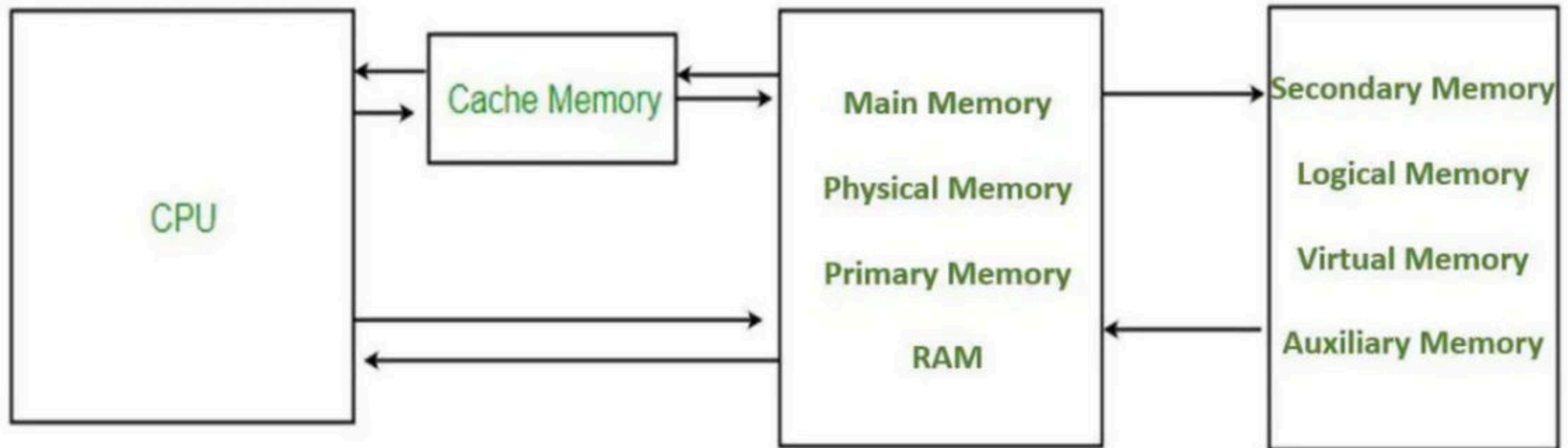
- If we want this hierarchy to work fine, then most of the time when CPU needs a data it must be present in Cache, if not possible main memory, worst case Secondary memory.
- But this is a difficult task to do as my computer has, 8 TB of Secondary Memory, 32 GB of Main Memory, but only 768KB, 4MB, 16 MB of L_1 , L_2 , L_3 cache respectively.
- If somehow we can estimate what data CPU will require if future we can prefetch in Cache and Main Memory.
- Locality of reference Helps we to perform this estimation.



- The main memory occupies a central position by being able to communicate directly with the CPU.
- When programs not residing in main memory are needed by the CPU, they are brought in from auxiliary memory.
- Programs not currently needed in main memory are transferred into auxiliary memory to provide space for currently used programs and data.

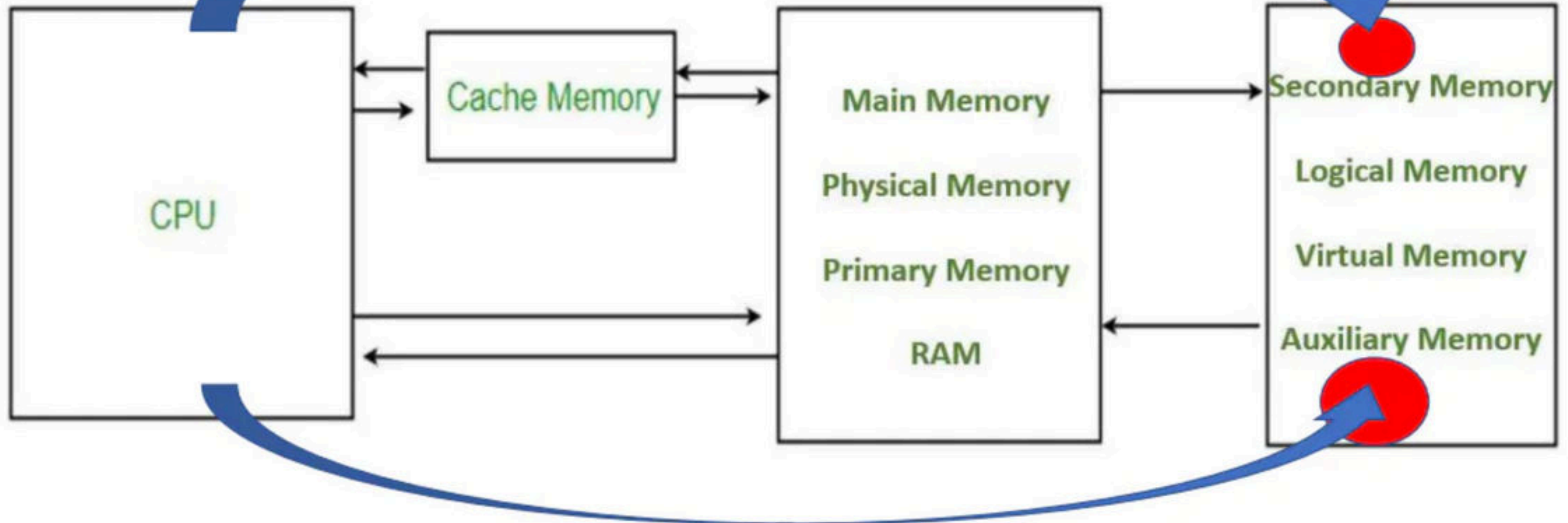


- A special very-high speed memory called a Cache is used to increase the speed of processing by making current programs and data available to the CPU at a rapid rate.
- The cache memory is employed in computer systems to compensate for the speed differential between main memory access time and processor logic.

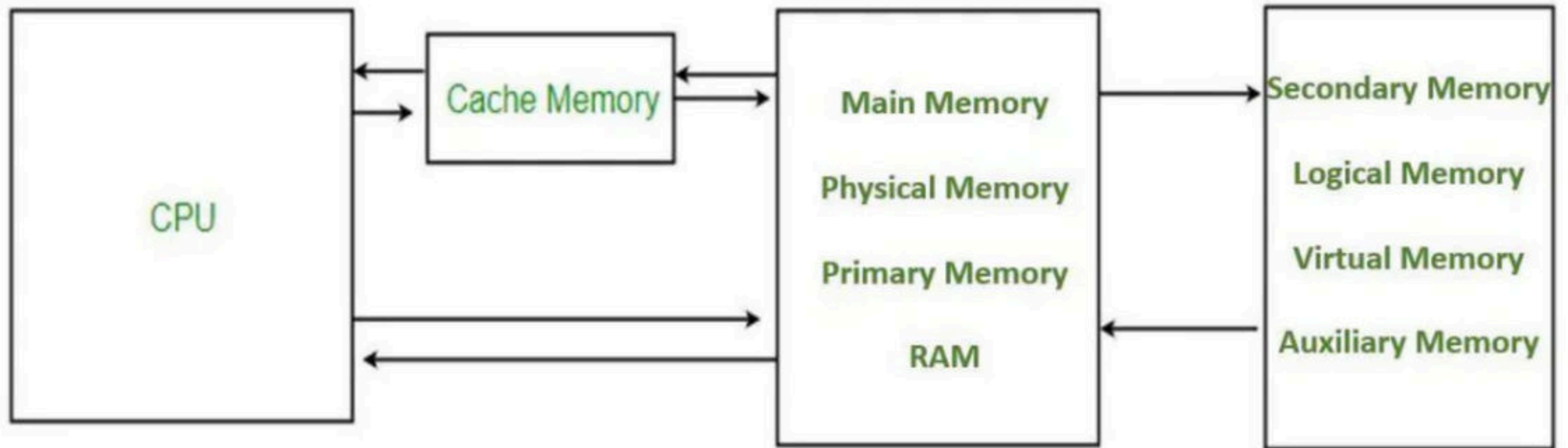


Locality of Reference

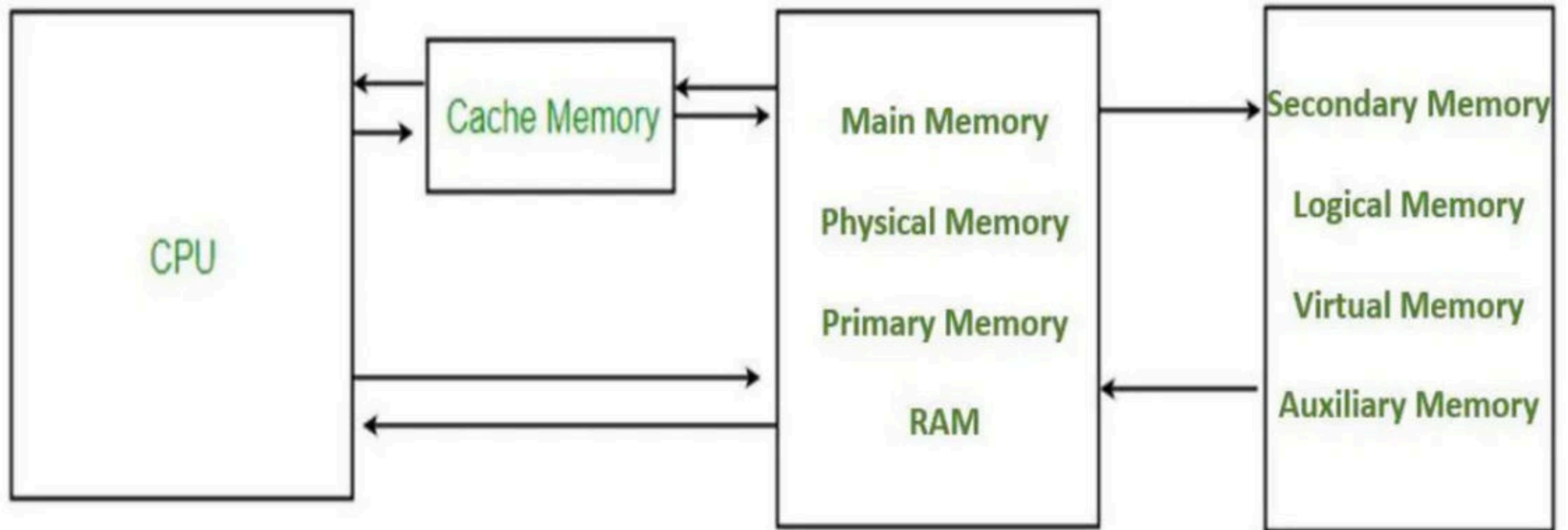
- The references to memory at any given interval of time tend to be confined within a few localized areas in memory. This phenomenon is known as the property of locality of reference. There are two types of locality of reference



- **Spatial Locality:** Spatial locality refers to the use of data elements in the nearby locations.



- **Temporal Locality:** Temporal locality refers to the reuse of specific data, and/or resources, within a relatively small-time duration. Or, the most frequently used items will be needed soon. (LRU is used for temporal locality)





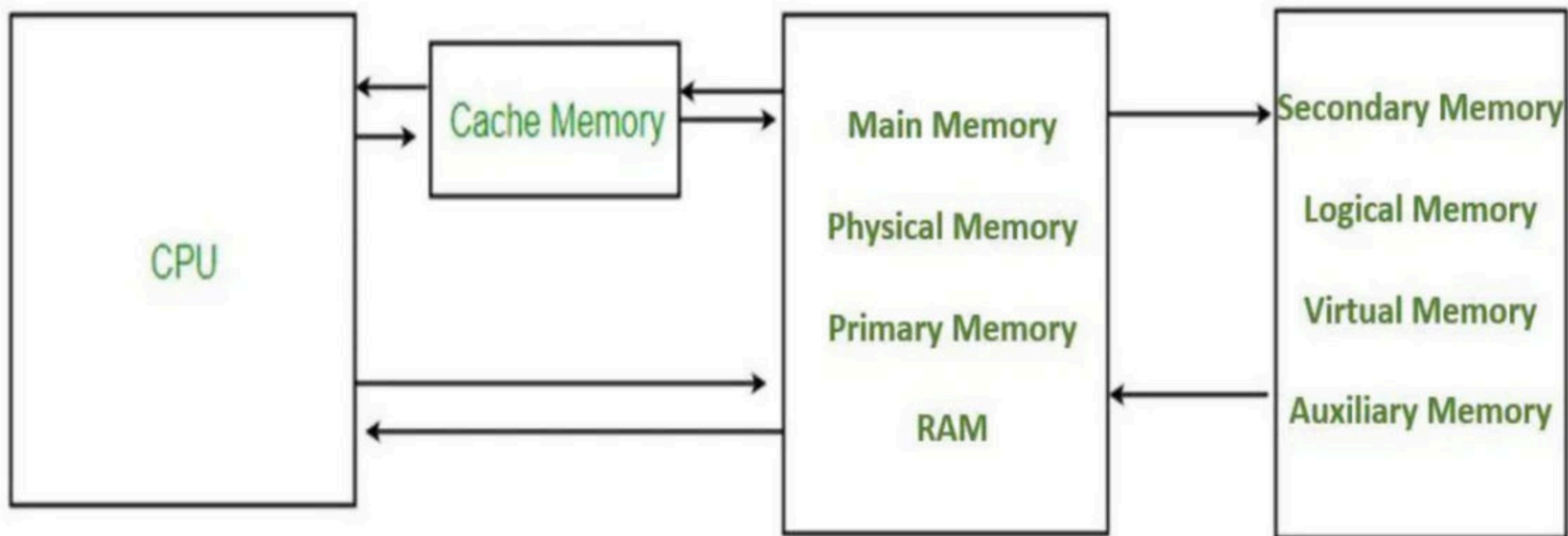
Showroom



Go down



Factory



- Reason to have memory hierarchy instead of a single memory is
 - Average access time (Should be less)
 - Capacity (Should be high)
 - Cost Per unit (Should be less)

Break

Duty of Operating System

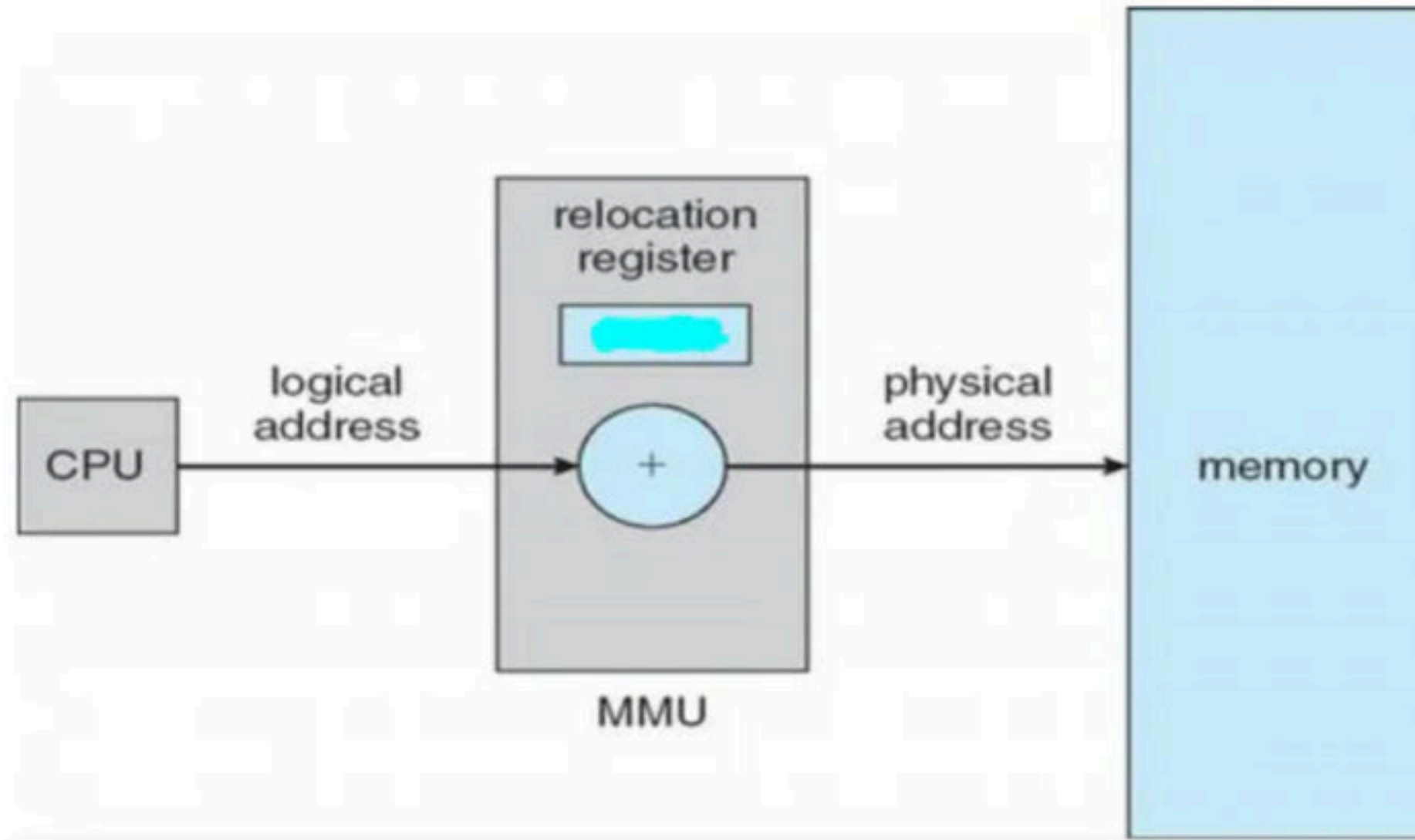
- Operating system is responsible for the following activities in connection with memory management:
 - Address translation from Logical address to Physical address.
 - Deciding which processes (or parts of processes) and data to move into and out of memory. Allocating and deallocating memory space as needed.
 - Keeping track of which parts of memory are currently being used and who is using them.

- The first duty of OS is to translate this logical address into physical address. There can be two approaches for storing a process in main memory.
 - Contiguous allocation policy
 - Non-contiguous allocation policy

Contiguous allocation policy

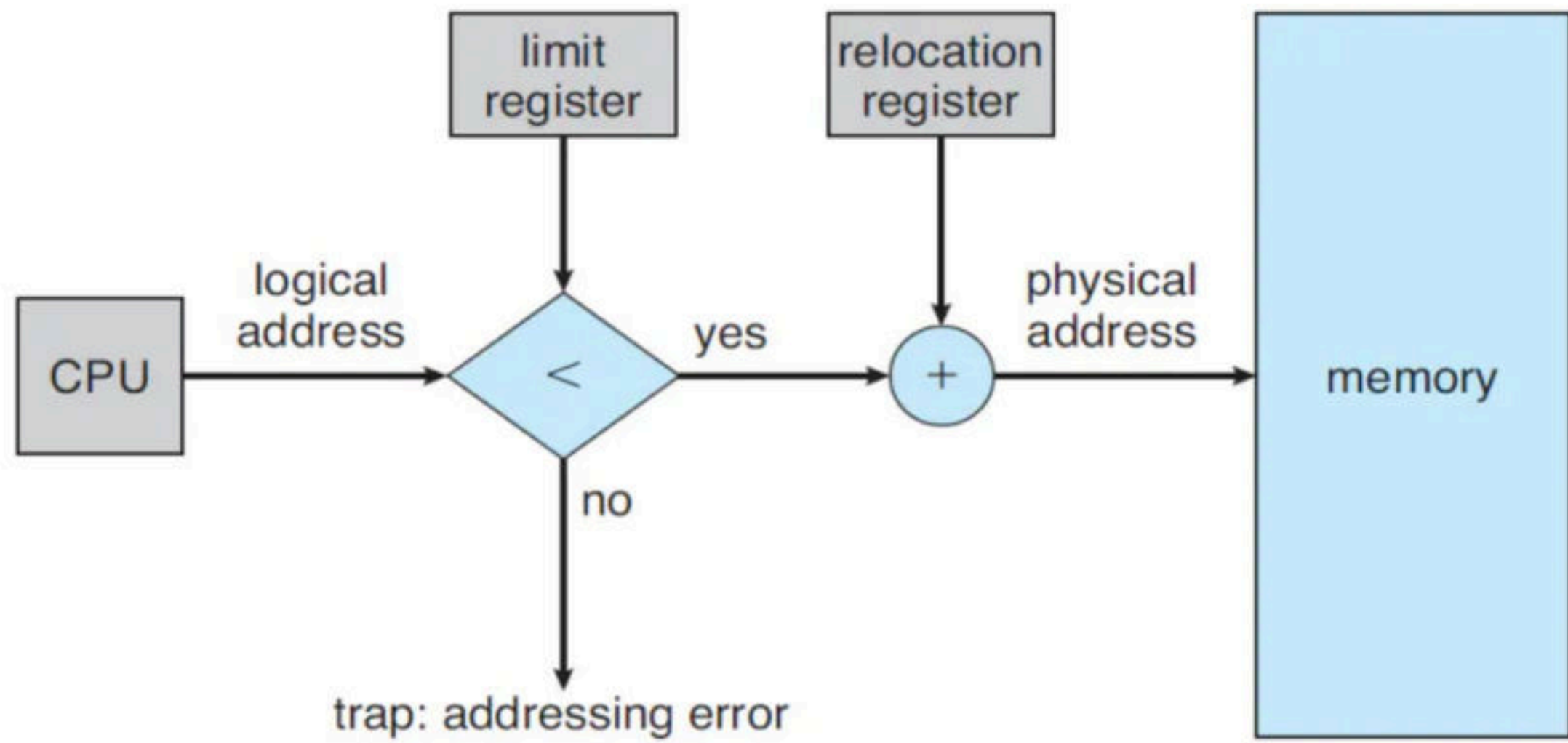
- We know that when a process is required to be executed it must be loaded to main memory, by policy has two implications.
 - It must be loaded to main memory completely for execution.
 - Must be stored in main memory in contiguous fashion.

- Here we use a memory management unit(OS) whose duty is to take logical address and translate it into physical address.
- MMU contains a relocation register, which contains the base address of the process in the main memory and it is added in the logical address every time



- **Advantage**: - easy strategy, simple to use, simple to understand.
- **Disadvantage**: - if a process generates logical address greater than its maximum limit, then it can easily access the data of some other process.

- In order to check whether address generated to CPU is valid(with in range) or invalid, we compare it with the value of limit register, which contains the max no of instructions in the process.
- So, if the value of logical address is less than limit, then it means it's a valid request and we can continue with translation otherwise, it is a illegal request which is immediately trapped by OS.



LA	PA
0	$0 + r$
Max	$\text{Max} + r$
Min	$\text{Min} + r$

Q Consider the following tables of relocation and limit registers and find the illegal attempts and if valid find the physical address?

	Limit Register	Relocation Register
P_0	500	1200
P_1	275	550
P_2	212	880
P_3	420	1400
P_4	118	200

Following are the request by process

P_0 —450

P_1 —300

P_2 —210

P_3 —450

P_4 ---80

Break

Space Allocation Method in Contiguous Allocation

- **Variable size partitioning:** -In this policy, in starting, we treat the memory as a whole or a single chunk & whenever a process request for some space, exactly same space is allocated if possible and the remaining space can be reused again.

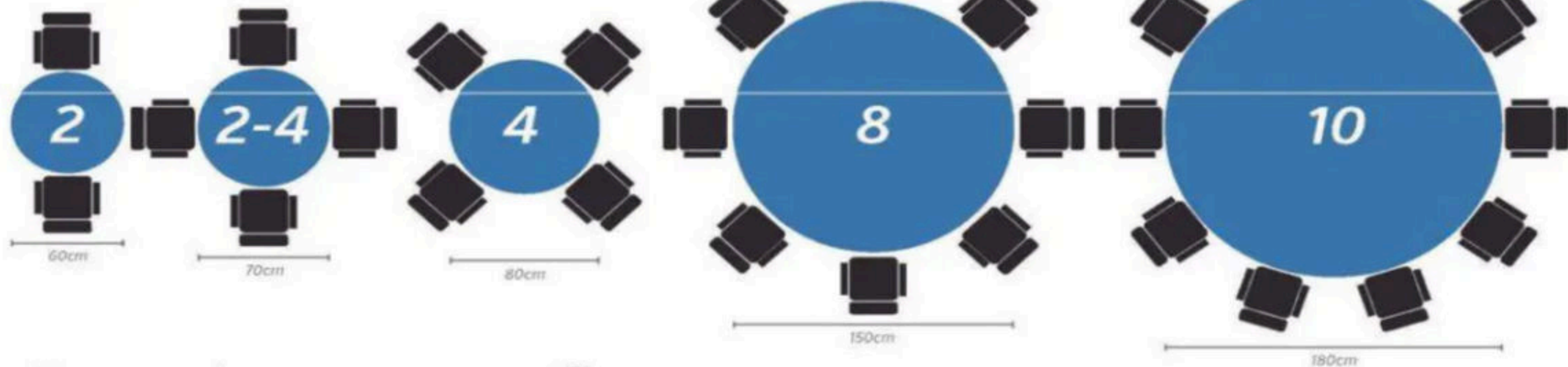




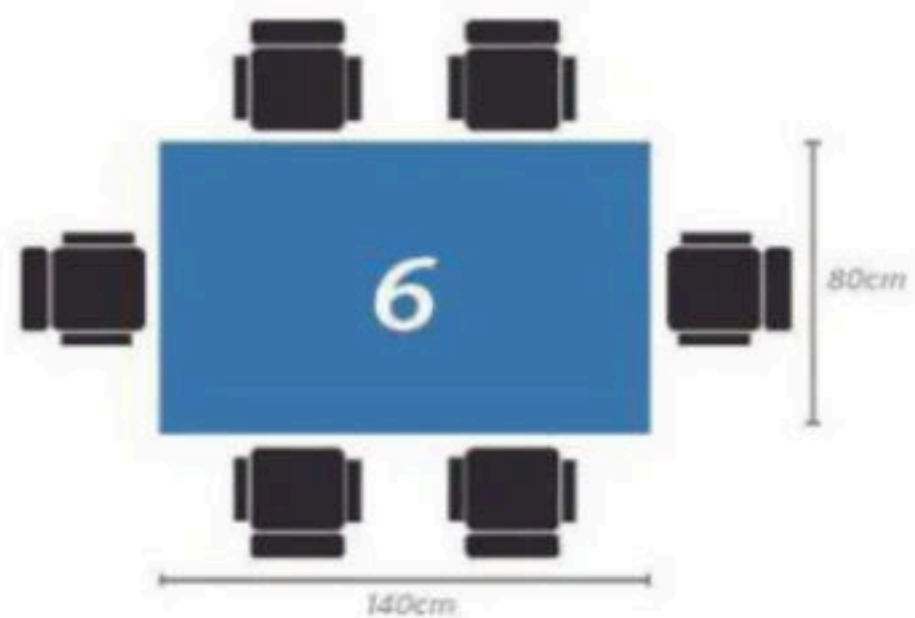
- **Fixed size partitioning:** - here, we divide memory into fixed size partitions, which may be of different sizes, but here if a process request for some space, then a partition is allocated entirely if possible, and the remaining space will be wasted internally.



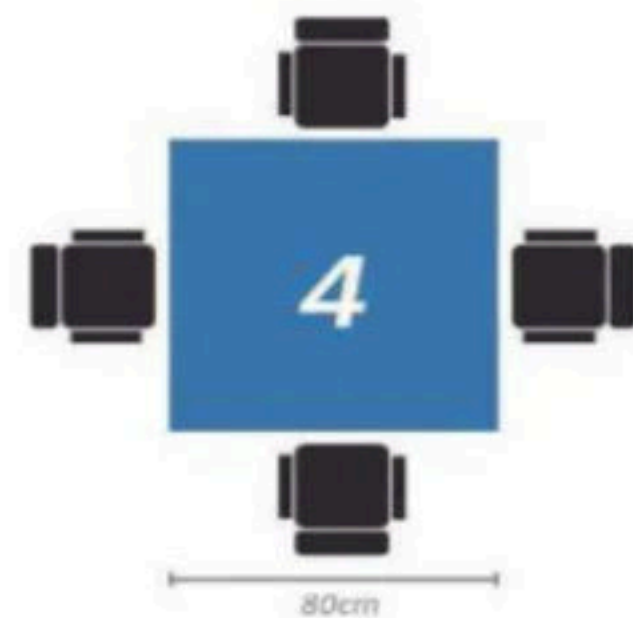
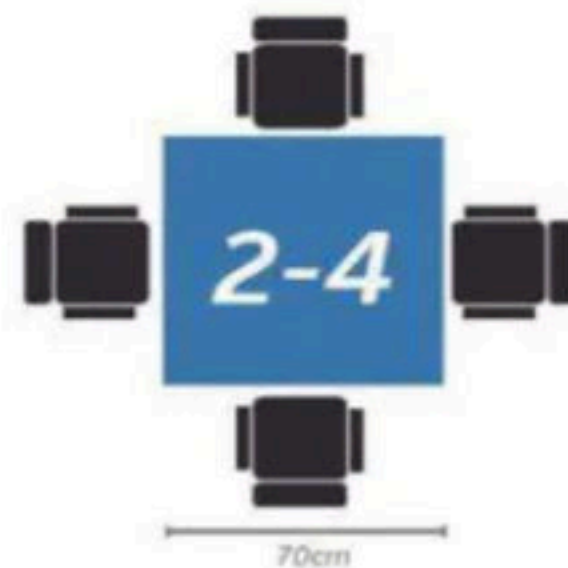
Circle



Rectangle



Square



- **First fit policy:** - as the name implies, it states searching the memory from the base and will allocate first partition which is capable enough.
 - **Advantage:** - simple, easy to use, easy to understand
 - **Disadvantage:** -poor performance, both in terms of time and space

--

--	--	--	--	--

- **Best fit policy:** - Here, we search the entire memory and will allocate the smallest partition which is capable enough.
 - **Advantage:** - perform best in fix size partitioning scheme.
 - **Disadvantage:** - difficult to implement, perform worst in variable size partitioning as the remaining spaces which are of very small size.



- **Worst fit policy:** - it also searches the entire memory and allocate the largest partition possible.
 - **Advantage:** - perform best in variable size partitioning
 - **Disadvantage:** - perform worst in fix size partitioning, resulting into large internal fragmentation.



- **Next fit policy:** - next fit is the modification in the best fit where, after satisfying a request, we start satisfying next request from the current position.

--

--	--	--	--	--

- **External fragmentation:** - External fragmentation is a function of contiguous allocation policy. The space requested by the process is available in memory but, as it is not being contiguous, cannot be allocated this wastage is called external fragmentation.

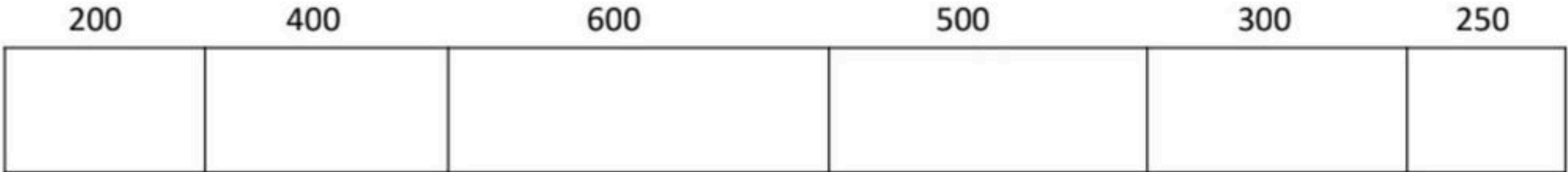
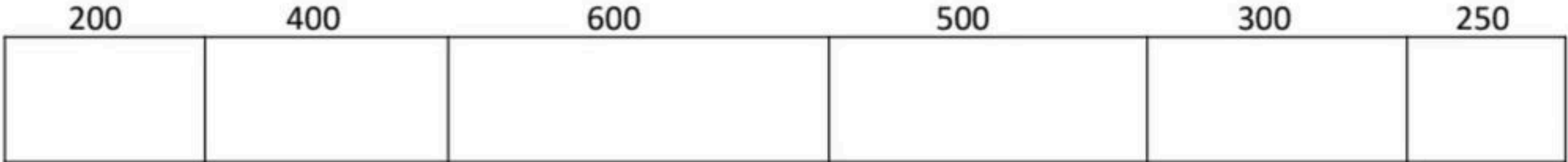
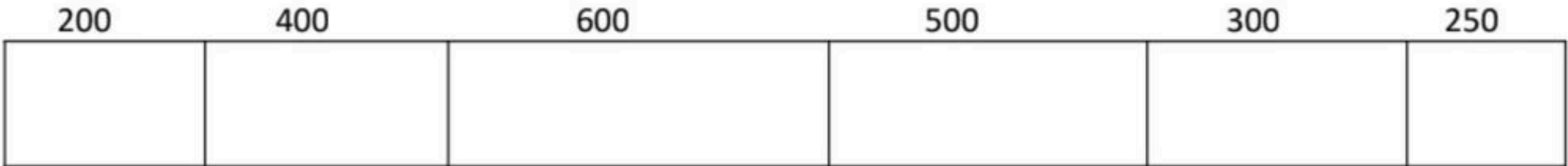


The Big Cason Family Are Expecting Their 17th Child

- **Internal fragmentation:** - Internal fragmentation is a function of fixed size partition which means, when a partition is allocated to a process. Which is either the same size or larger than the request then, the unused space by the process in the partition is called as internal fragmentation



Q Consider six memory partitions of size 200 KB, 400 KB, 600 KB, 500 KB, 300 KB, and 250 KB, where KB refers to kilobyte. These partitions need to be allotted to four processes of sizes 357 KB, 210 KB, 468 KB and 491 KB in that order. If the best fit algorithm is used, which partitions are NOT allotted to any process? **(GATE-2015) (2 Marks)**
(A) 200 KB and 300 KB **(B)** 200 KB and 250 KB **(C)** 250 KB and 300 KB **(D)** 300 KB and 400 KB



Q Consider the following heap (figure) in which blank regions are not in use and hatched region are in use. (GATE-1994) (2 Marks)

The sequence of requests for blocks of size 300, 25, 125, 50 can be satisfied if we use

- (a) either first fit or best fit policy (any one)
- (b) first fit but not best fit policy
- (c) best fit but first fit policy
- (d) None of the above



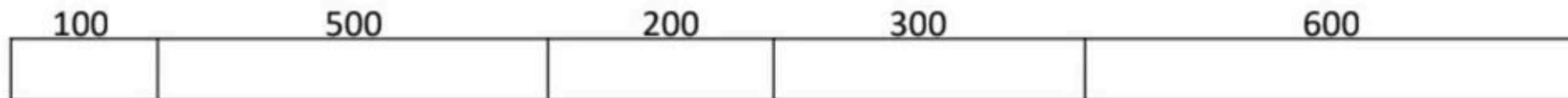
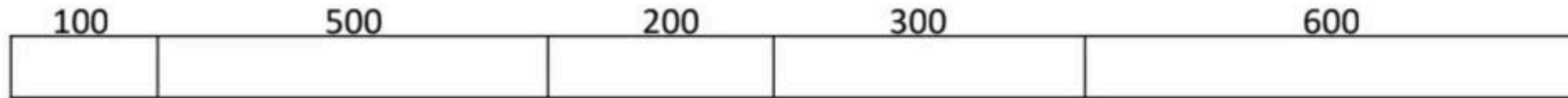
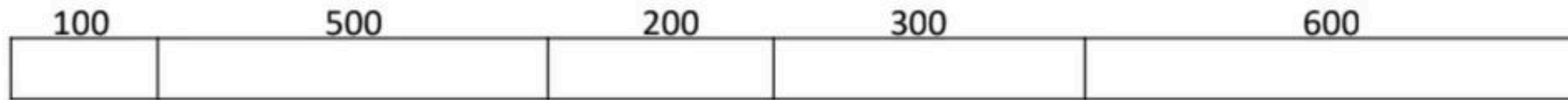
Q Given memory partitions of 100 K, 500 K, 200 K, 300 K and 600 K (in order) and processes of 212 K, 417 K, 112 K, and 426 K (in order), using the first-fit algorithm, in which partition would the process requiring 426 K be placed ? **(NET-DEC-2012)**

(A) 500 K

(B) 200 K

(C) 300 K

(D) 600 K



Use Referral Code KGYT for Unacademy Plus to Get minimum 10% Discount

Q A program is located in the smallest available hole in the memory is
(NET-DEC-2009)

- (A)** best – fit **(B)** first – bit **(C)** worst – fit **(D)** buddy

Q A 1000 Kbyte memory is managed using variable partitions but No compaction. It currently has two partitions of sizes 200 Kbytes and 260 Kbytes respectively. The smallest allocation request in Kbytes that could be denied is for? **(GATE-1996) (1 Marks)**

(a) 151

(b) 181

(c) 231

(d) 541

- How can we solve external fragmentation?
 - We can also swap processes in the main memory after fixed intervals of time & they can be swapped in one part of the memory and the other part become empty (Compaction, defragmentation). This solution is very costly in respect to time as it will take a lot of time to swap process when system is in running state.
 - Either we should go for non-contiguous allocation, which means process can be divided into parts and different parts can be allocated in different areas.

Q Variable partition memory management technique with compaction results in:
(NET-JUNE-2009)

(A) Reduction of fragmentation

(B) Minimal wastage

(C) Segment sharing

(D) None of the above

Q Which of the following statements are true ? **(NET-JULY-2018)**

(a) External Fragmentation exists when there is enough total memory space to satisfy a request but the available space is contiguous.

(b) Memory Fragmentation can be internal as well as external.

(c) One solution to external Fragmentation is compaction.

Code :

(a) (a) and (b) only

(b) (a) and (c) only

(c) (b) and (c) only

(d) (a), (b) and (c)

Use Referral Code KGYT for Unacademy Plus to Get minimum 10% Discount

Q In which of the following storage replacement strategies, is a program placed in the largest available hole in the memory? **(NET-DEC-2004)**

(A) Best fit

(B) First fit

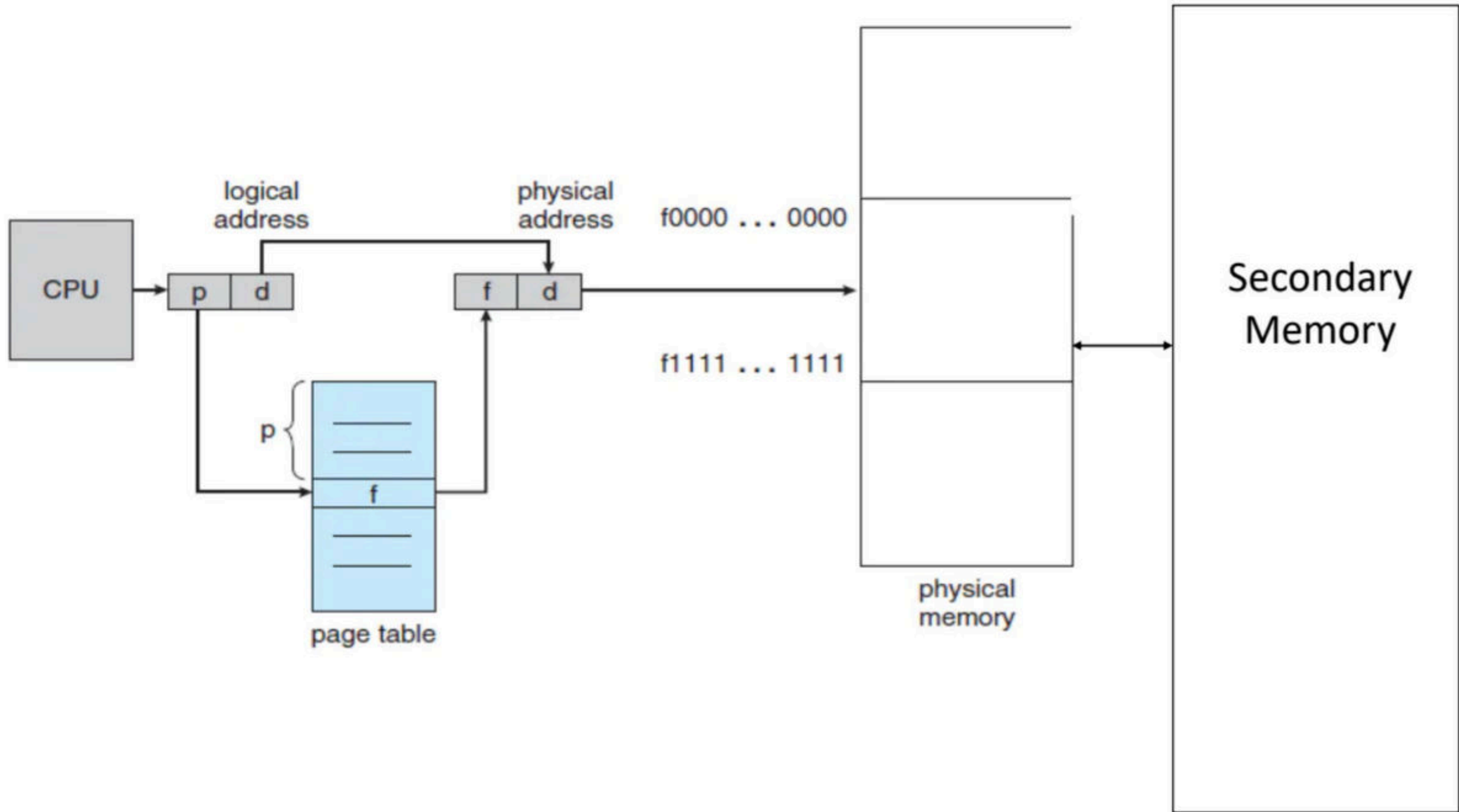
(C) Worst fit

(D) Buddy

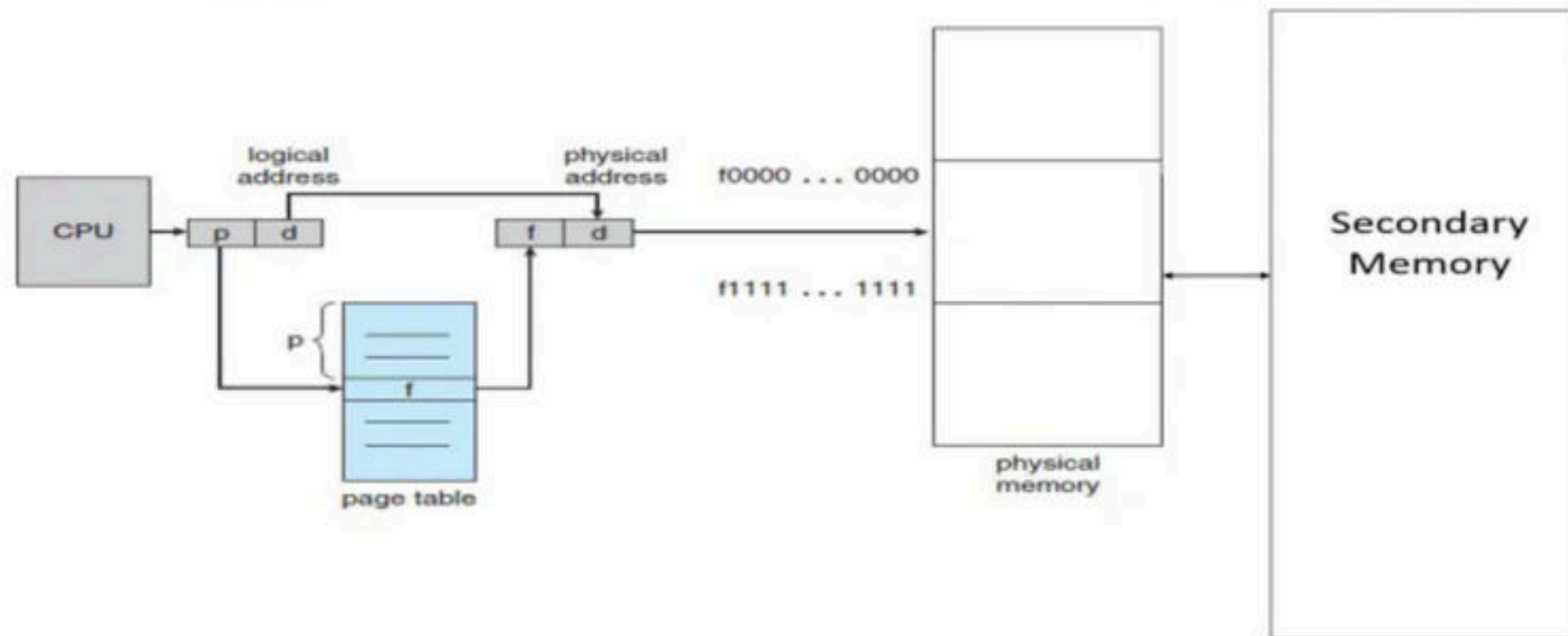
Break

Non-Contiguous Memory allocation(Paging)

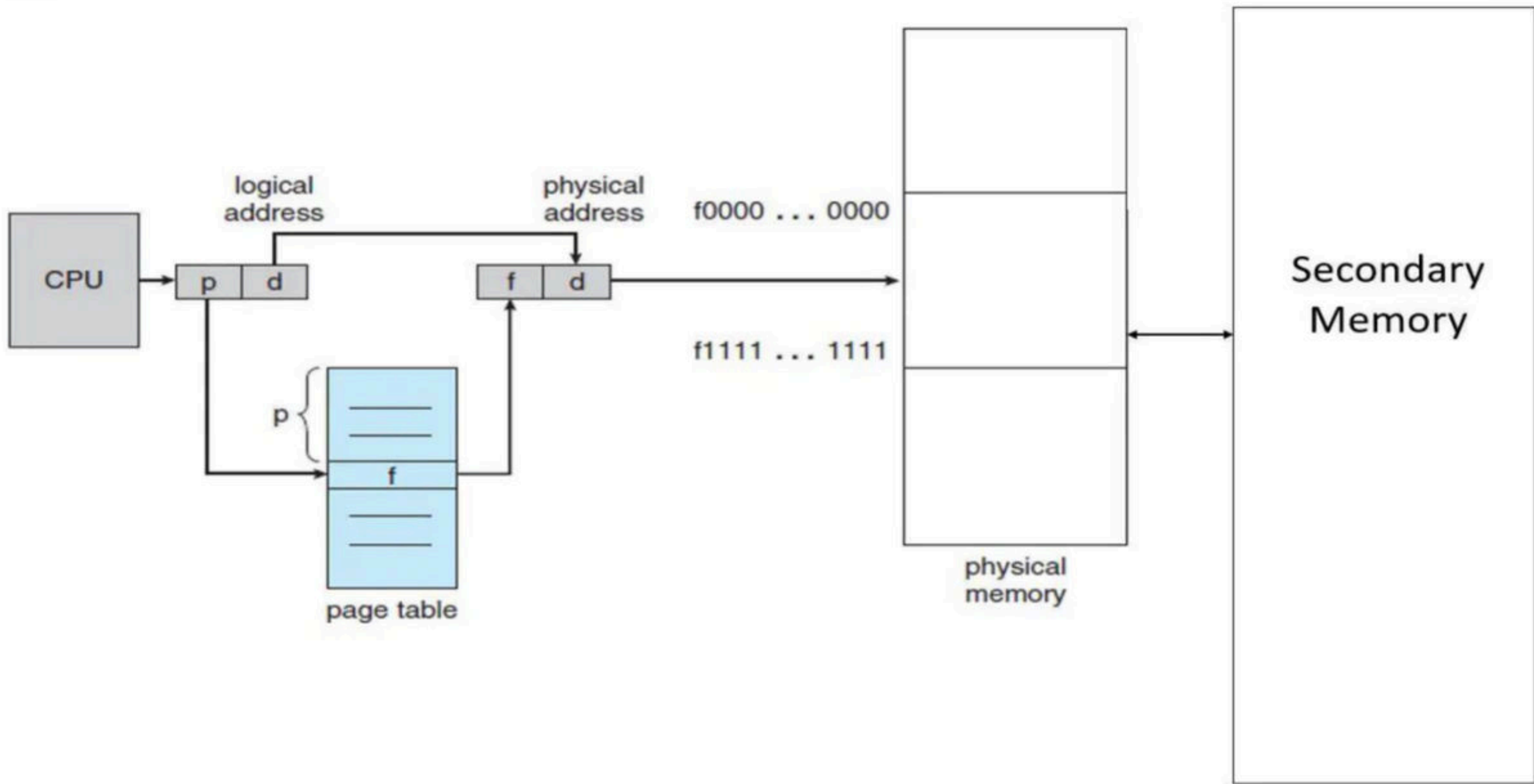
- Paging is a memory-management scheme that permits the physical address space of a process to be non-contiguous.
- Paging avoids external fragmentation

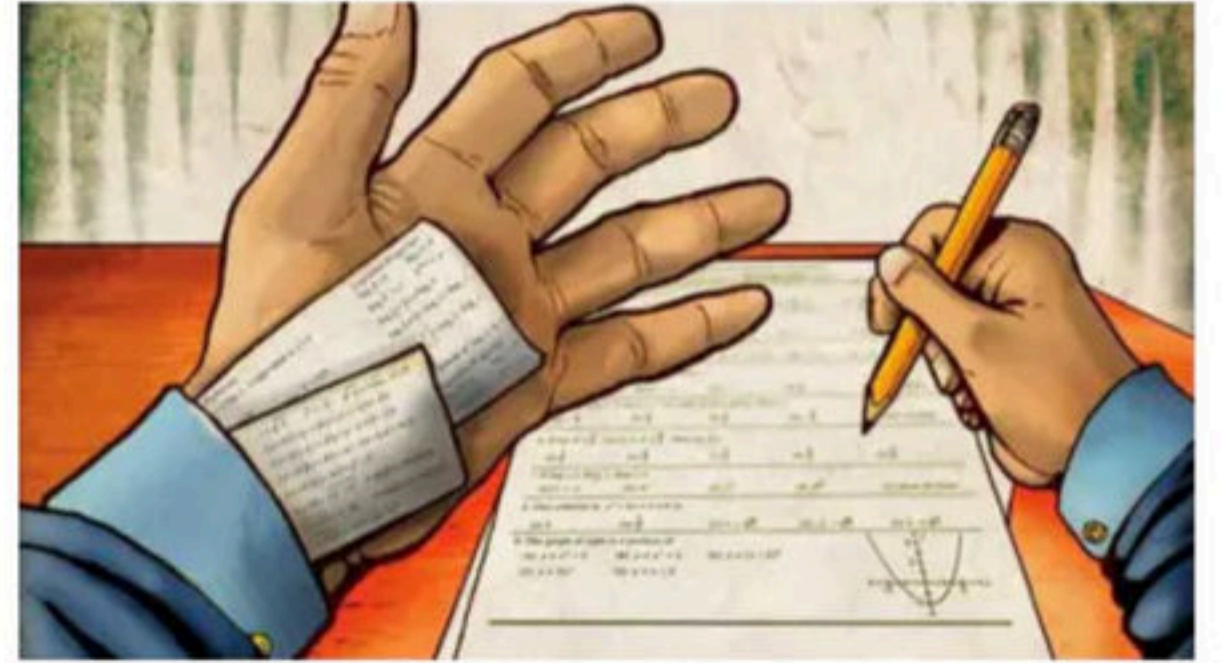
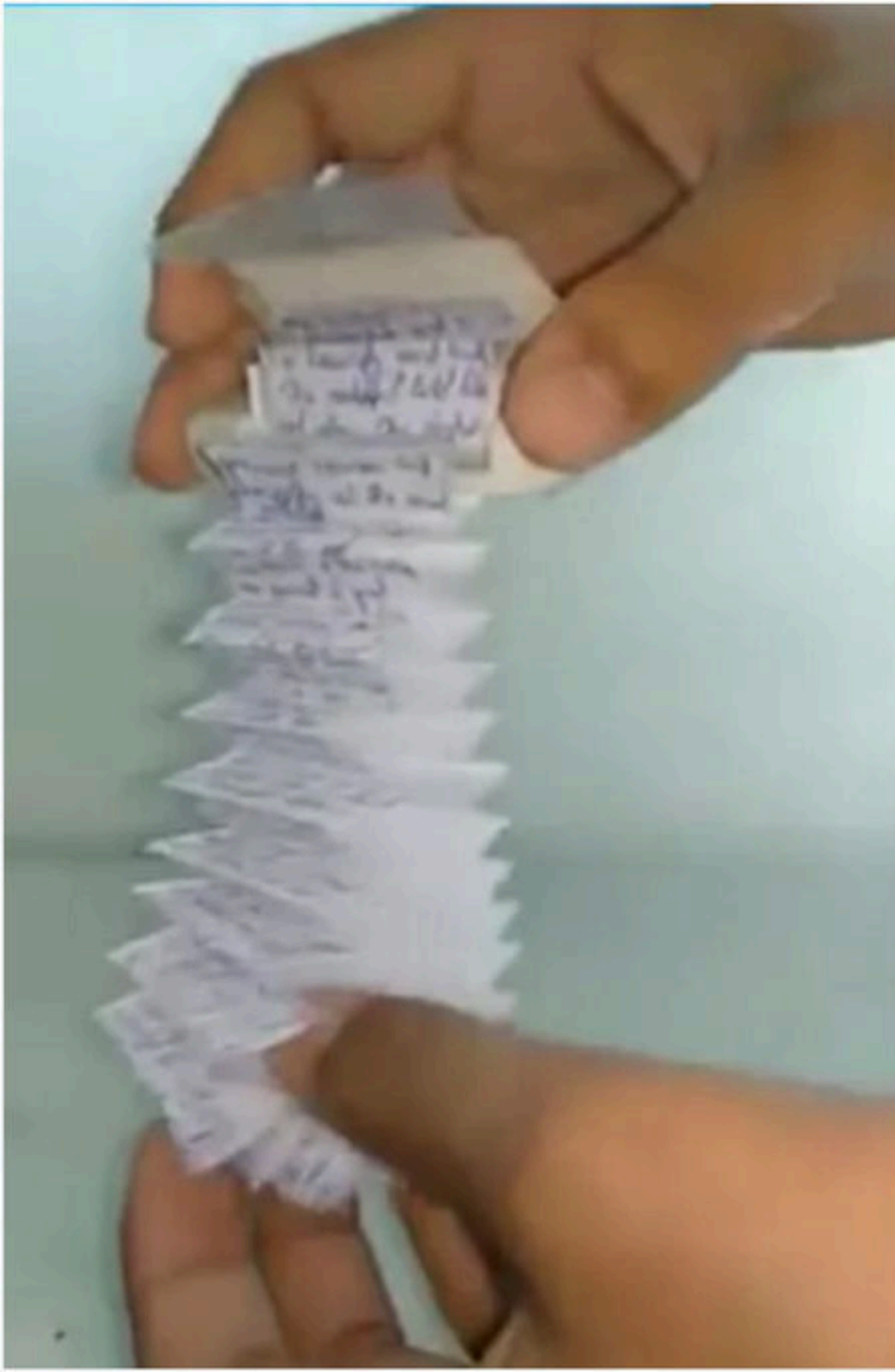


- Secondary memory is divided into fixed size partition(because management is easy) all of them of same size called pages(easy swapping and no external fragmentation).
- Main memory is divided into fix size partitions (because management is easy), each of them having same size called frames(easy swapping and no external fragmentation).
- Size of frame = size of page
- In general number of pages are much more than number of frames (approx. 128 time)



Break





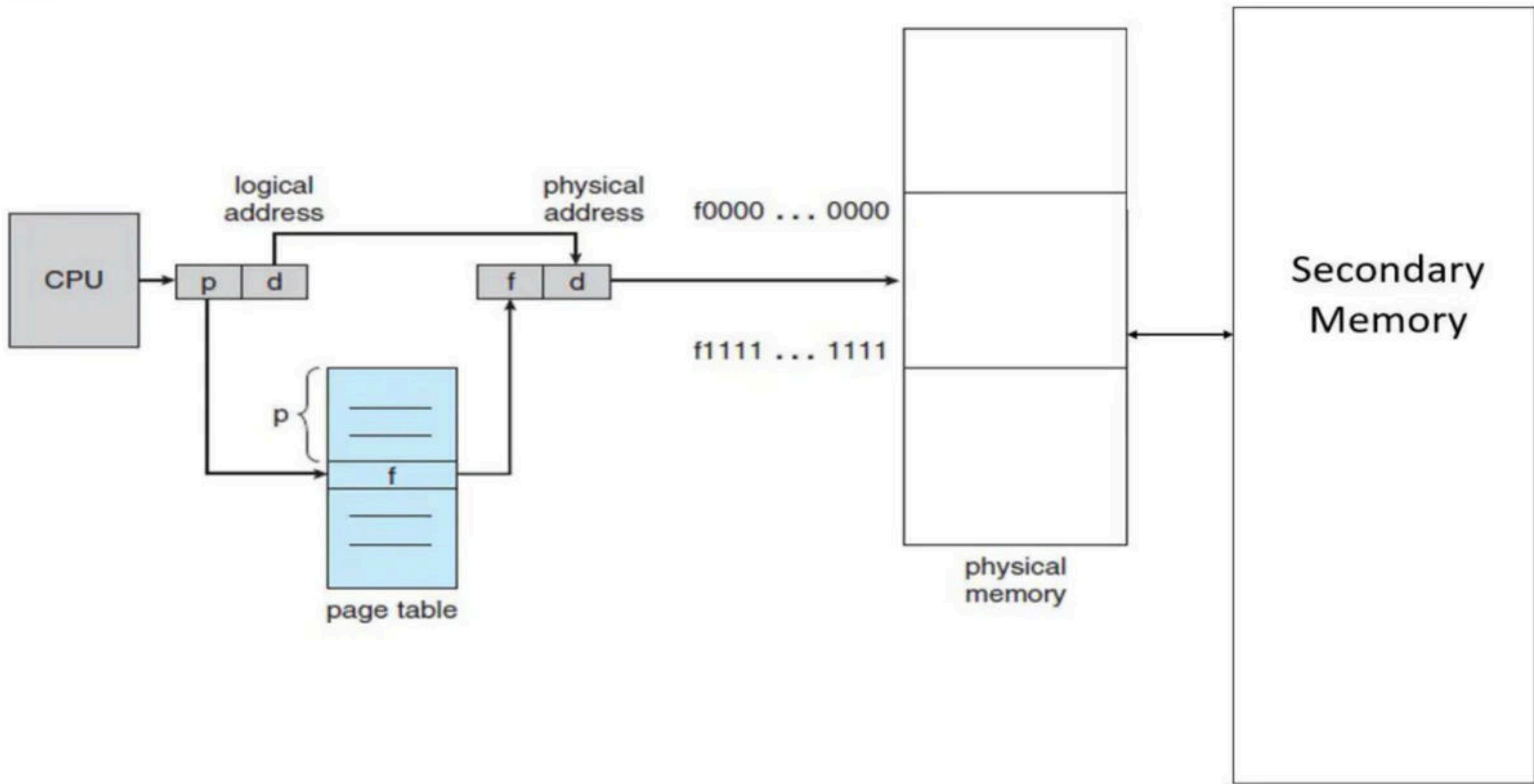
Translation process

1. CPU generate a logical address is divided into two parts - **p** and **d**
 1. where p stands for page no and d stands for instruction offset.
2. The **page number(p)** is used as an index into a **Page table**
3. **Page table base register(PTBR)** provides the base of the page table and then the corresponding page no is accessed using p.
4. Here we will finds the corresponding frame no (the base address of that frame in main memory in which the page is stored)
5. Combine corresponding frame no with the instruction offset and get the physical address. Which is used to access main memory.

Break

Page Table

1. Page table is a data structure not hardware.
2. Every process have a separate page table.
3. Number of entries a process have in the page table is the number of pages a process have in the secondary memory.
4. Size of each entry in the page table is same it is corresponding frame number.
5. Page table is a data structure which is it self stored in main memory.



Advantage

- Removal of External Fragmentation

Disadvantage

- Translation process is slow as Main Memory is accessed two times(one for page table and other for actual access).
- A considerable amount of space is wasted in storing page table(meta data).
- System suffers from internal fragmentation(as paging is an example of fixed size partition).
- Translation process is difficult and complex to understand and implement.

Q Page information in memory is also called as Page Table. The essential contents in each entry of a page table is/are _____. **(NET-JULY-2018)**

(a) Page Access information

(b) Virtual Page number

(c) Page Frame number

(d) Both virtual page number and Page Frame Number

Q The essential content(s) in each entry of a page table is / are? (GATE-2009) (1 Marks)

(A) Virtual page number

(B) Page frame number

(C) Both virtual page number and page frame number

(D) Access right information

Break

10^3	1 Thousand
10^6	1 Million
10^9	1 Billion
10^{12}	1 Trillion

10^3	1 kilo
10^6	1 Mega
10^9	1 Giga
10^{12}	1 Tera
10^{15}	1 Peta
10^{18}	1 Exa
10^{21}	1 Zetta
10^{24}	1 Yotta

2^{10}	1 kilo
2^{20}	1 Mega
2^{30}	1 Giga
2^{40}	1 Tera
2^{50}	1 Peta
2^{60}	1 Exa
2^{70}	1 Zetta
2^{80}	1 Yotta

Address Length in bits	n
No of Locations	2^n



Memory Size = Number of Location * Size of each Location

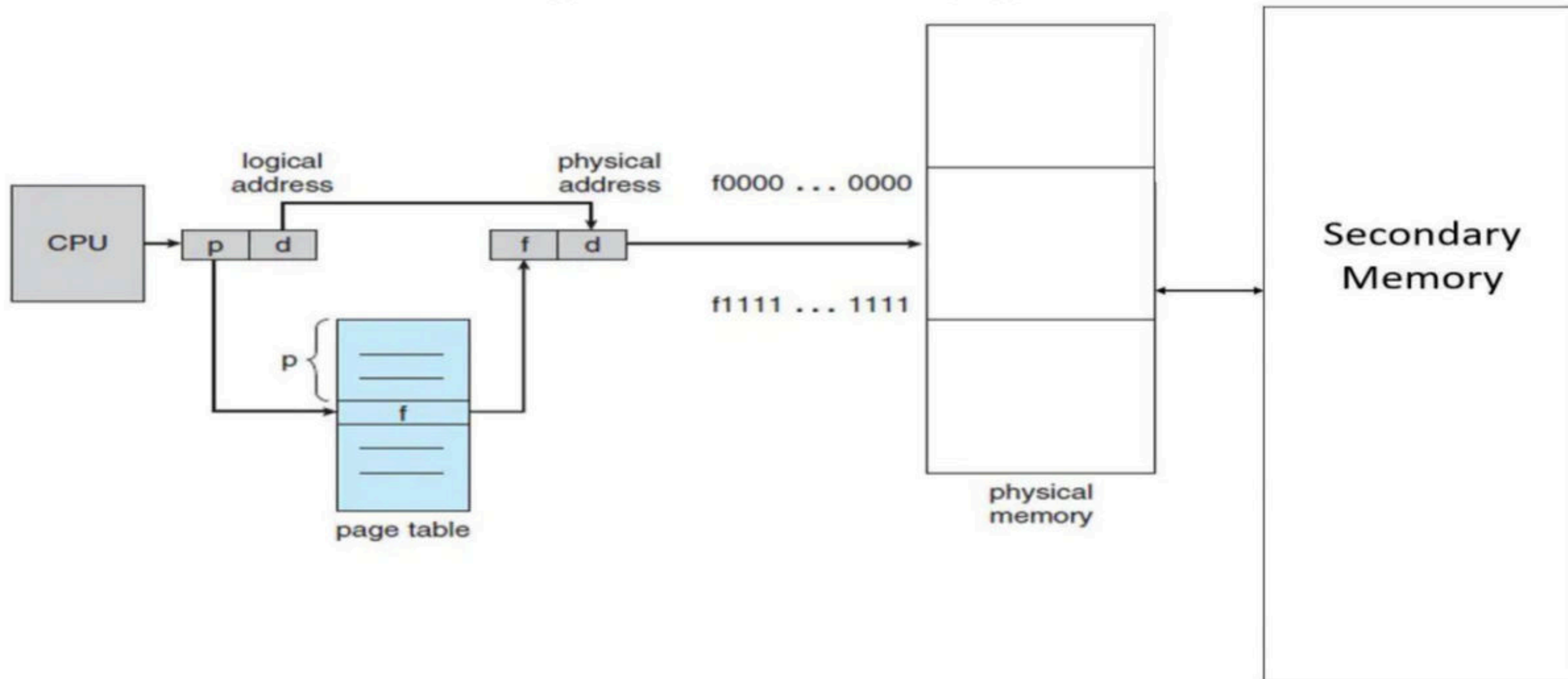
No of Locations	n
Address Length in bits	Upper Bound($\log_2 n$)



-
-
-
-
-
-

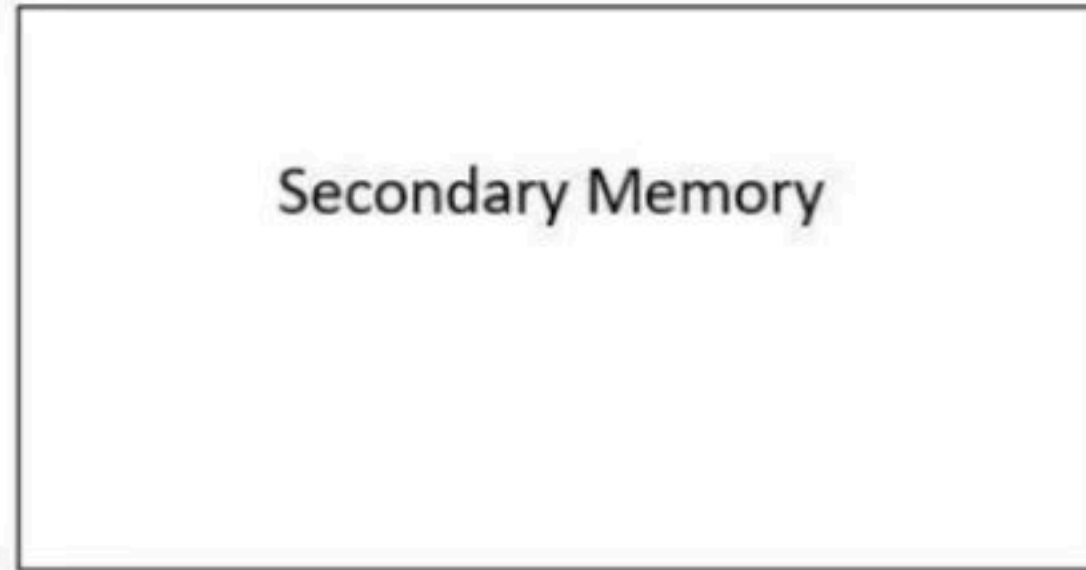
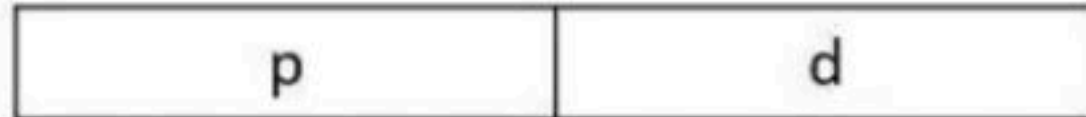
Memory Size/ Size of each Location = Number of Location

- Page Table Size = No of entries in Page table * Size of each entry(f)
- Process Size = No of Pages * Size of each page

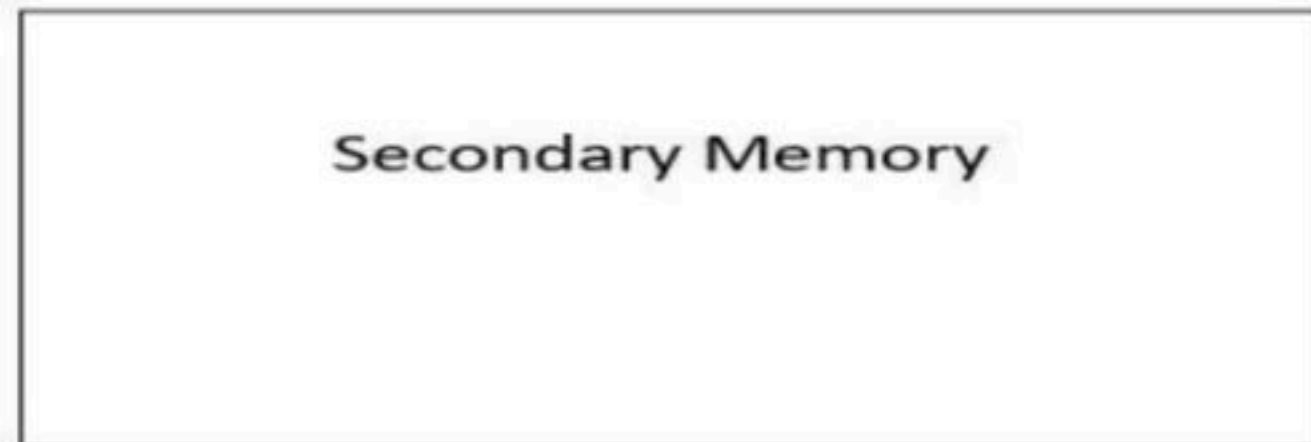


Logical Address in bits
Secondary Memory
Physical Address in bits
Main Memory

p
f
d



S No	SM	LA	MM	PA	p	f	d	addressable	Page Size
1	32 GB		128 MB					1B	1KB
2		42		33			11	1B	
3	512GB			31				1B	512B
4	128GB		32GB		30			1B	
5					28	14			4096B



Break

Q A memory management system has 64 pages with 512 bytes page size. Physical memory consists of 32 page frames. Number of bits required in logical and physical address are respectively (**NET-DEC-2016**)

a) 14 and 15

b) 14 and 29

c) 15 and 14

d) 16 and 32

Q Consider a system with byte-addressable memory, 32-bit logical addresses, 4 kilobyte page size and page table entries of 4 bytes each. The size of the page table in the system in megabytes is _____ **(GATE-2015) (2 Marks)**

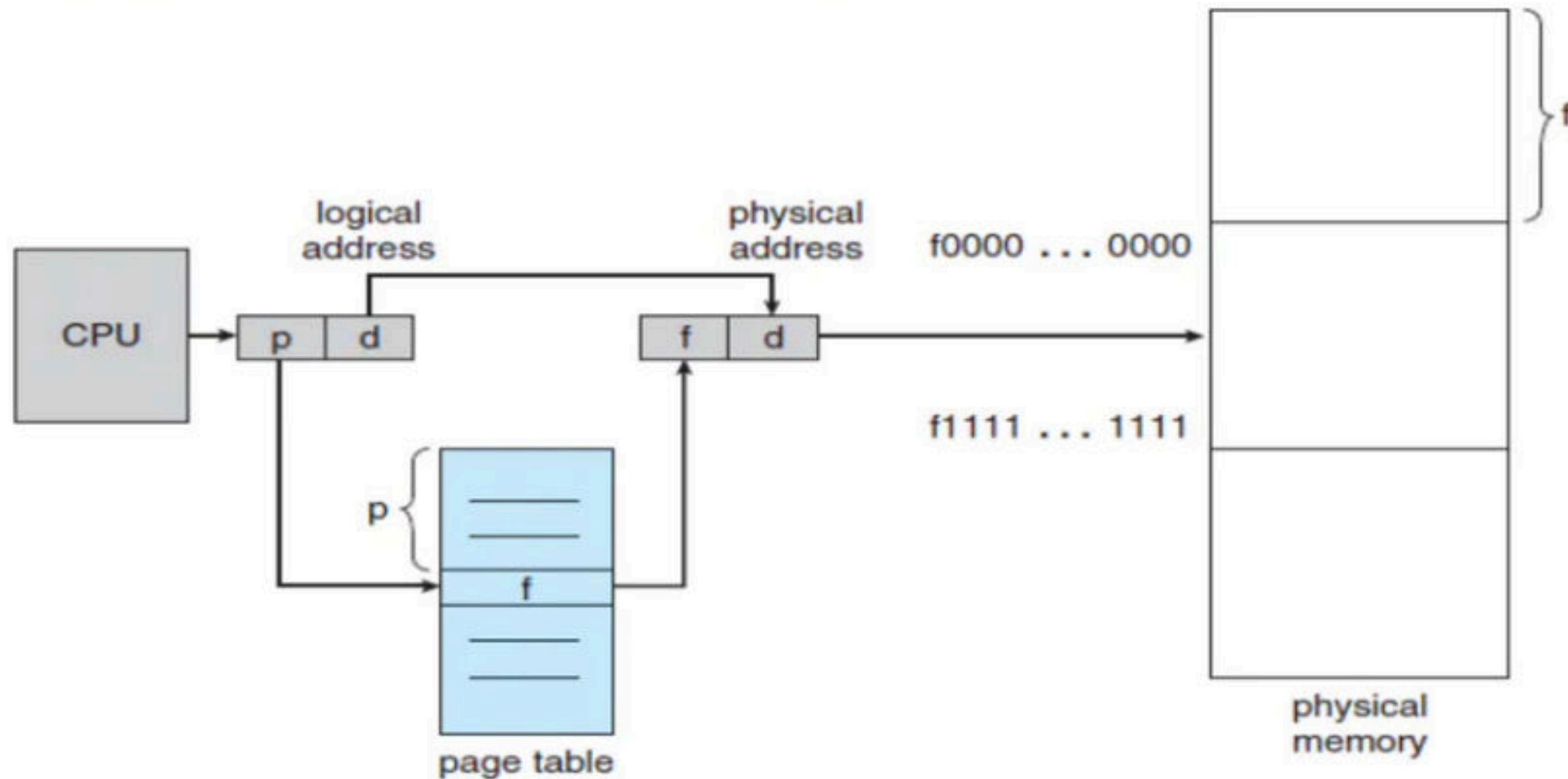


Figure 8.10 Paging hardware.

Q A Computer system implements 8 kilobyte pages and a 32-bit physical address space. Each page table entry contains a valid bit, a dirty bit three permission bits, and the translation. If the maximum size of the page table of a process is 24 megabytes, the length of the virtual address supported by the system is _____ bits? **(GATE-2015) (2 Marks)**

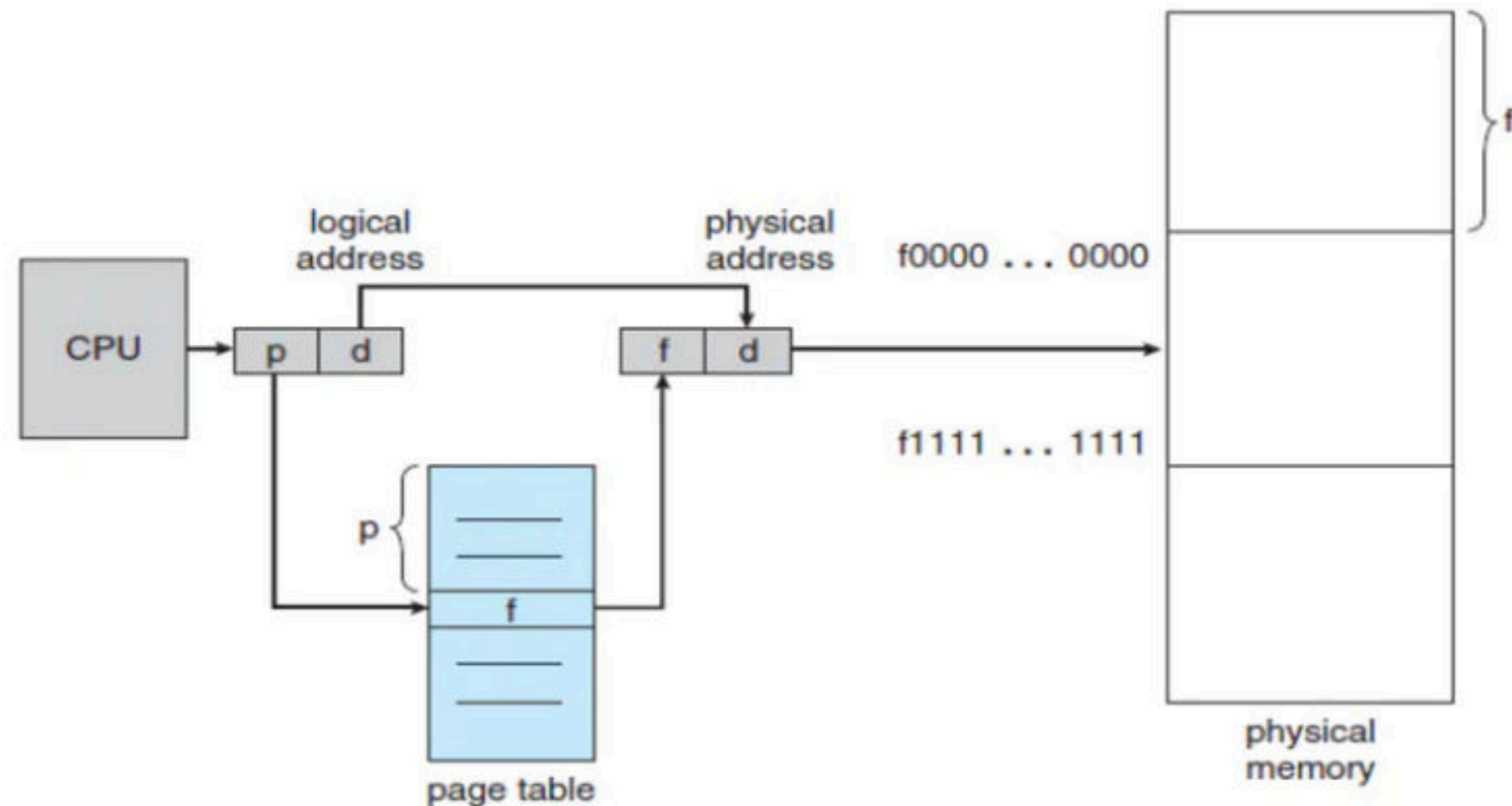


Figure 8.10 Paging hardware.

Q A computer system supports 32-bit virtual address as well as 32-bit physical addresses. Since the virtual address space is of same size as that of physical address space, if we want to get rid of virtual memory, which one of the following is true? **(NET-JUNE-2012)**

(A) Efficient implementation of multiuser support is no longer possible.

(B) The processor cache can be made more efficient.

(C) Hardware support for memory management is not needed.

(D) CPU scheduling can be made more efficient.

Q Consider a logical address space of 8 pages of 1024 words mapped with memory of 32 frames. How many bits are there in the physical address? **(NET-DEC-2011)**

(A) 9 bits

(B) 11 bits

(C) 13 bits

(D) 15 bits

Q Consider a machine with 64 MB physical memory and a 32-bit virtual address space. If the page size is 4KB, what is the approximate size of the page table? **(GATE-2001) (2 Mark)**

(a) 16 MB

(b) 8 MB

(c) 2 MB

(d) 24 MB

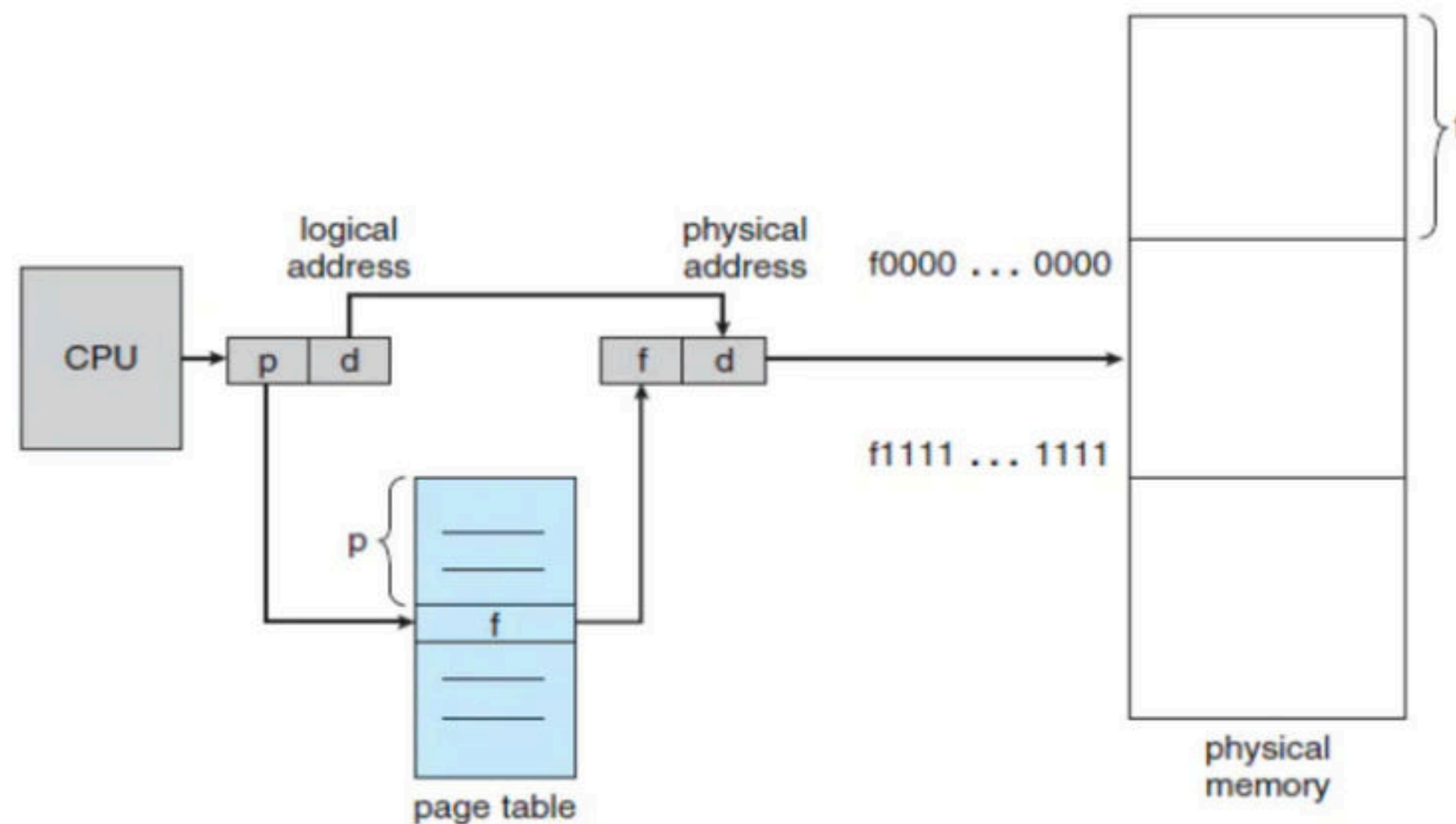


Figure 8.10 Paging hardware.

Break

S No	SM	LA	MM	PA	p	f	d	addressable	Process Size	PT Size
1	16 GB	34	64 MB	26	24	16	10	1B	1 GB	2 MB
2	1 TB	44	64 MB	36	32	24	12	1B	1 GB	64 KB
3	512 MB	39	256 MB	28	31	26	8	1B	512 MB	8 MB

$$512 \text{ MB} = \frac{2^{29} \text{ B}}{2^8 \text{ B}} = 2^m \times 3 \text{ B}$$

$$1 \text{ GB} = \frac{2^{30} \text{ B}}{2^4 \text{ B}} = 2^m \times 3 \text{ B}$$

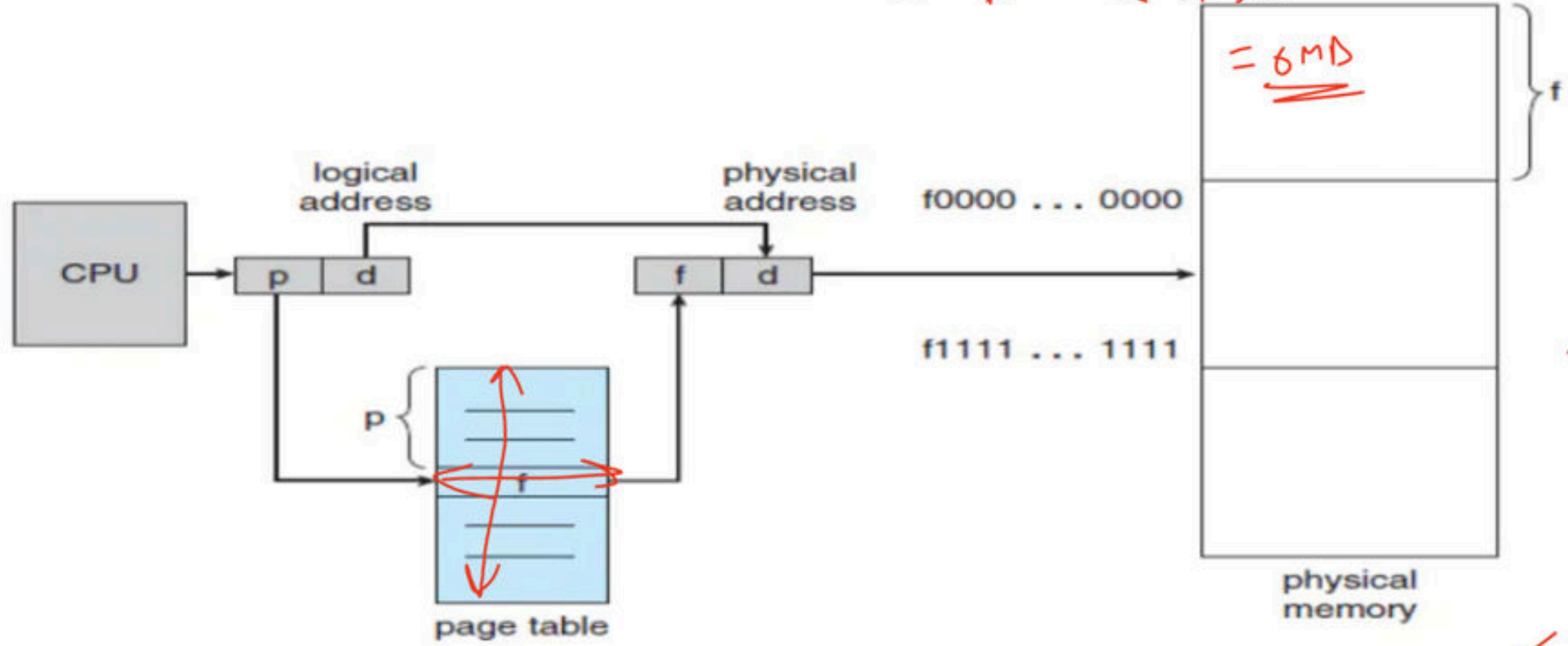


Figure 8.10 Paging hardware.

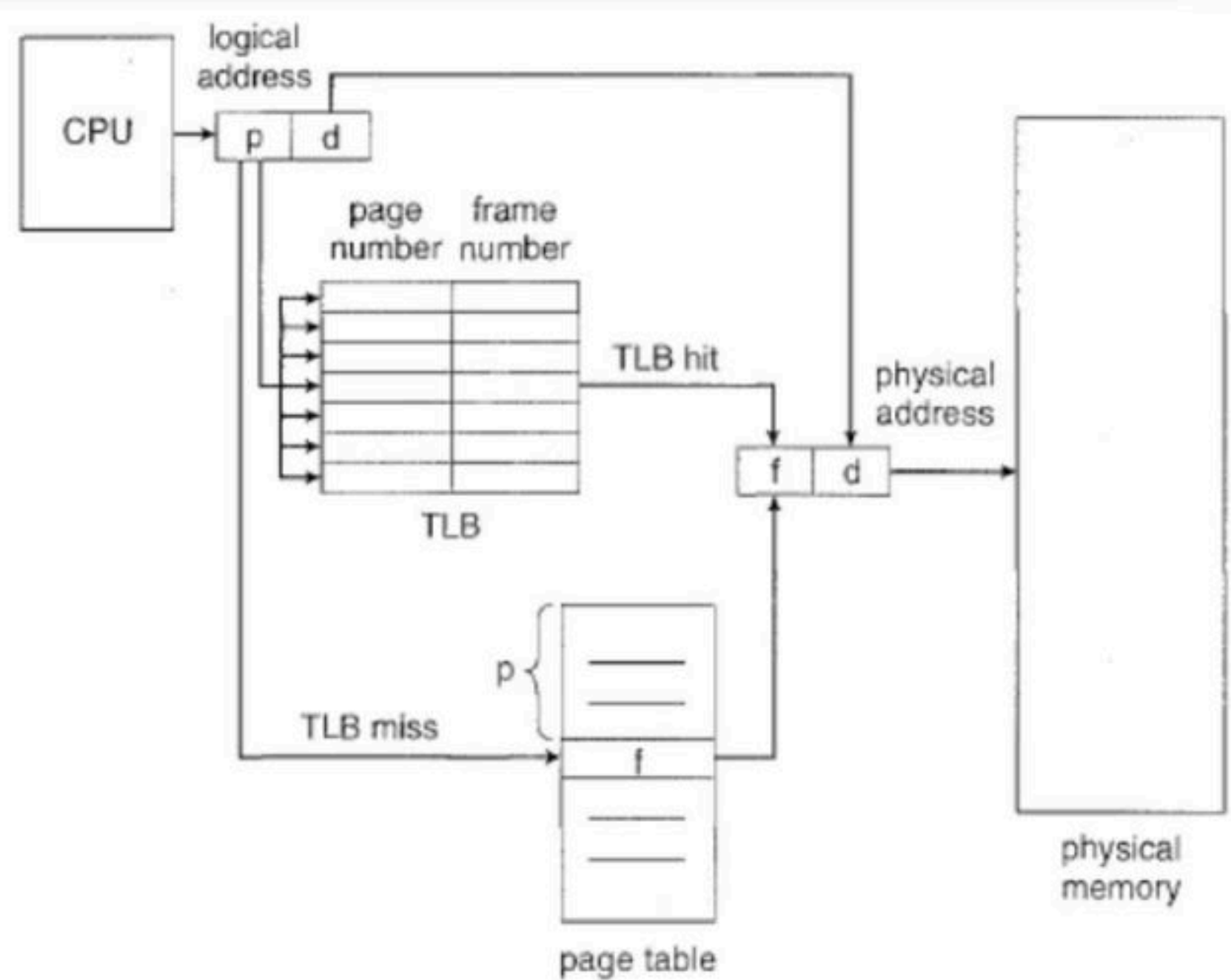
$$2 \text{ MB} = 2^m \times 3 \text{ B}$$

$$x = 1 \text{ M} = 2^{20}$$

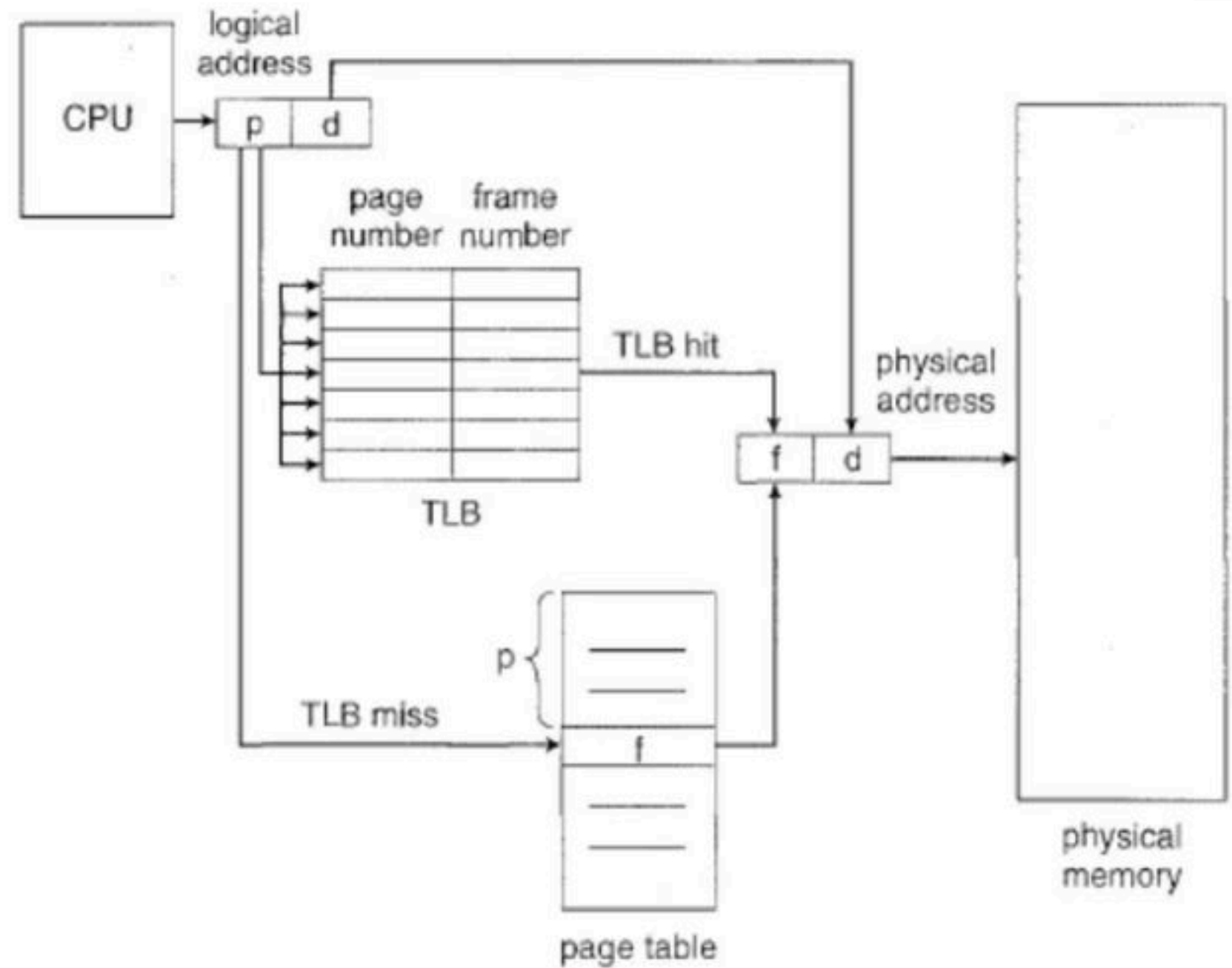
$$1 \text{ M} \times 16 \text{ B}$$

Break

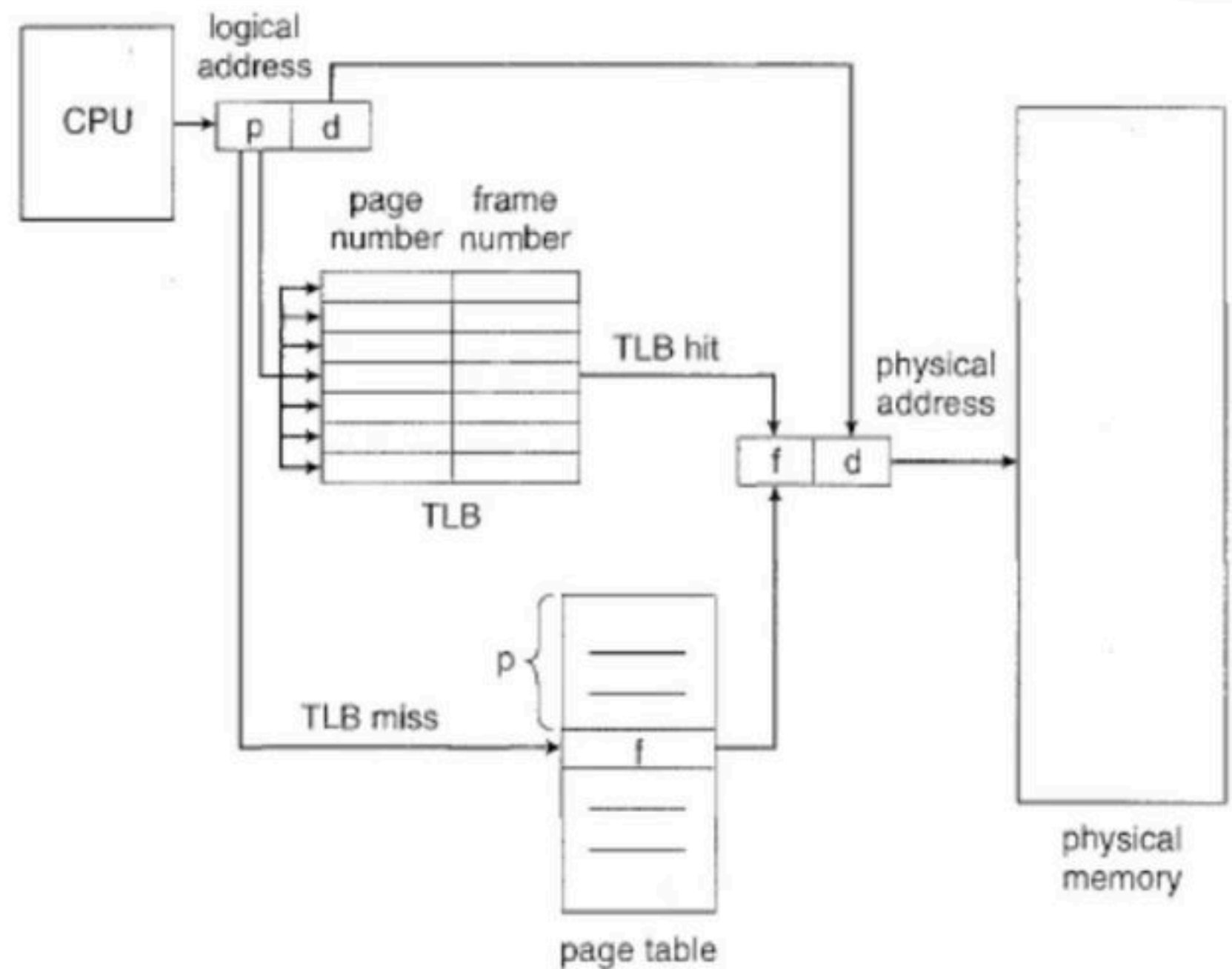
- A serious problem with page is, translation process is slow as page table is accessed two times (one for page table and other for actual access).
- To solve the problems in paging we take the help of TLB. The TLB is associative, high-speed memory.



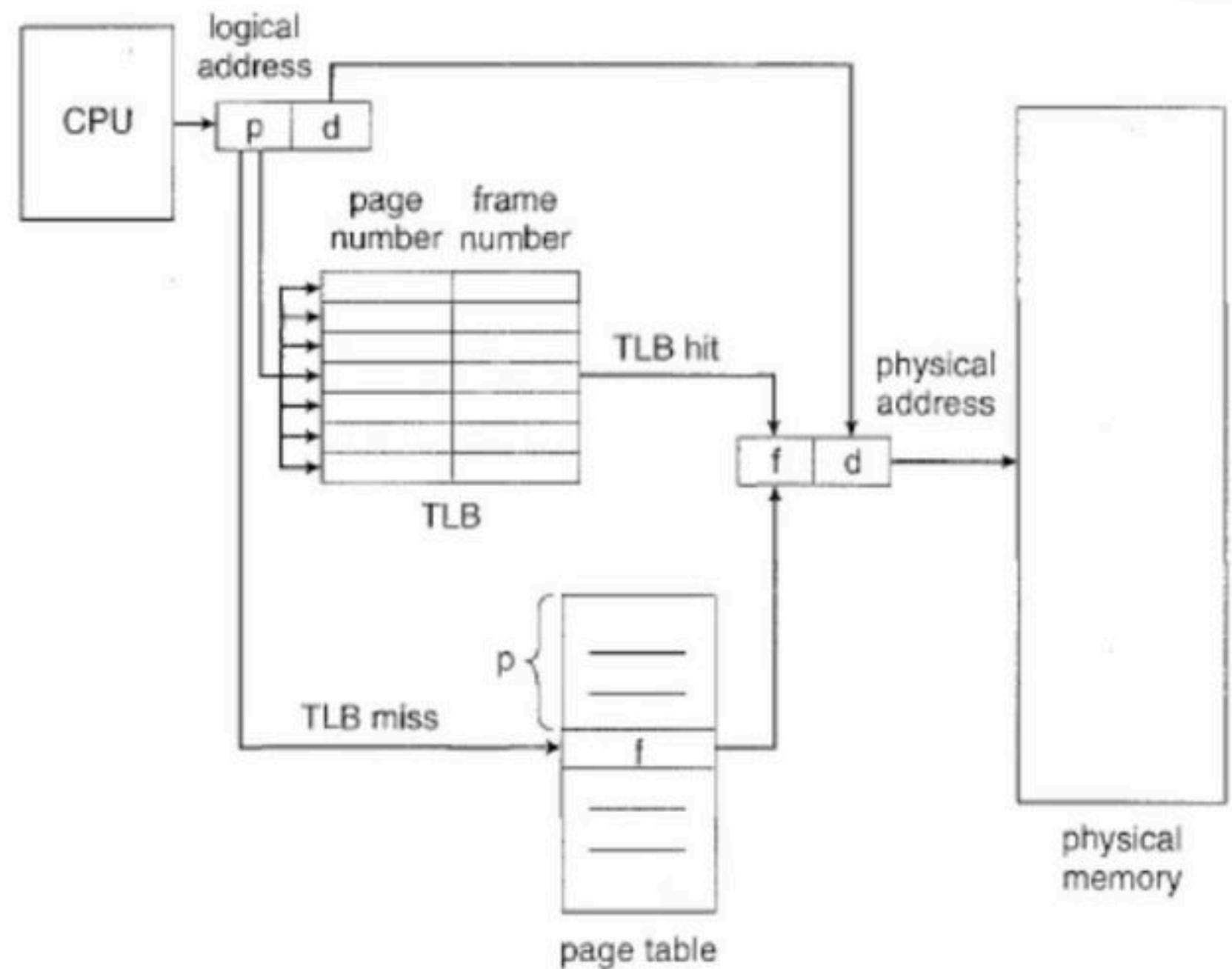
- Each entry in the TLB consists of two parts: a key (Page no) and a value (frame no). When the associative memory is search for page no, the page no is compared with all page no simultaneously. If the item is found, the corresponding frame no field is returned.
- The search is fast; **the hardware, however, is expensive**, TLB Contains the frequently referred page numbers and corresponding frame number.



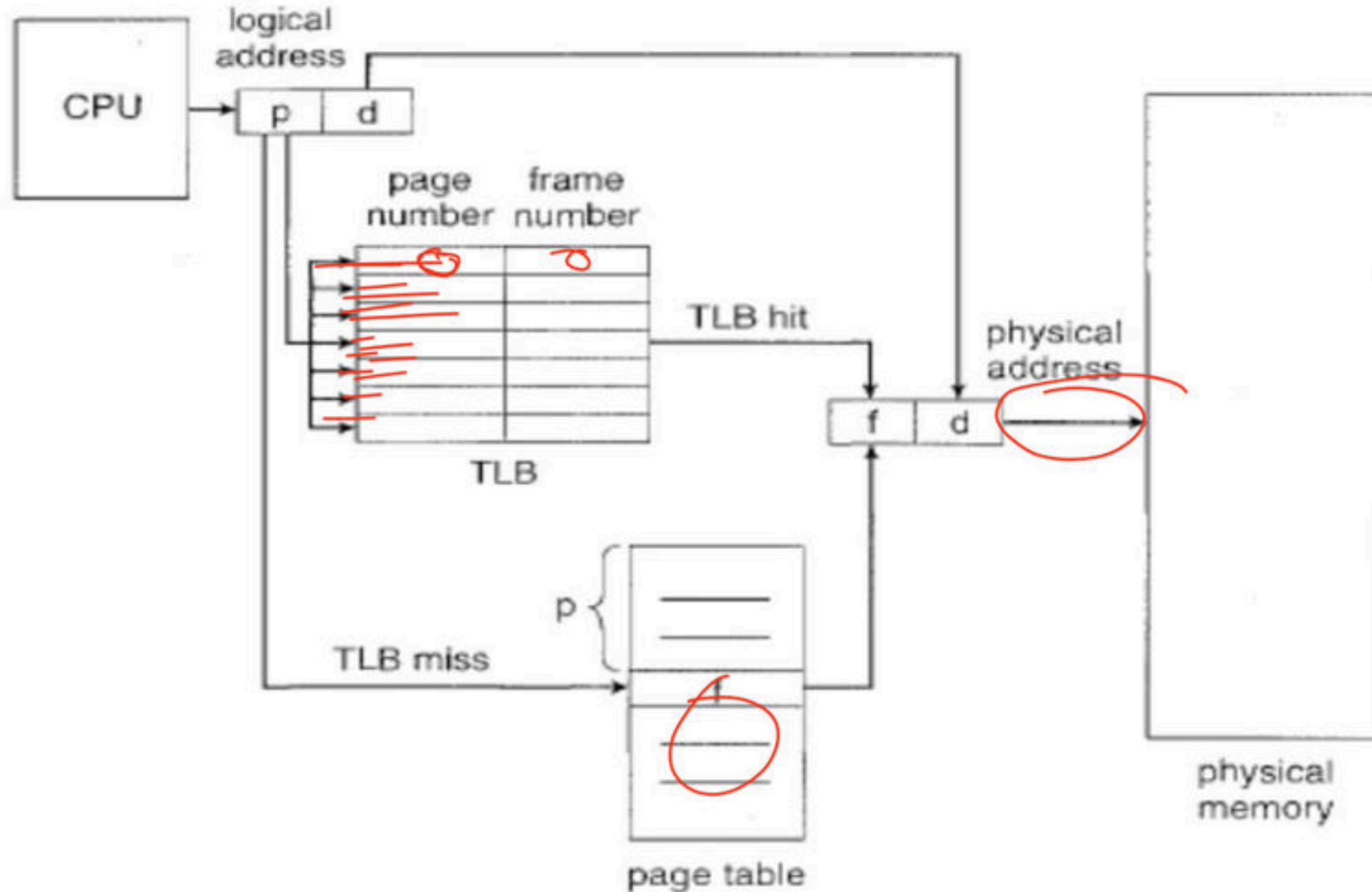
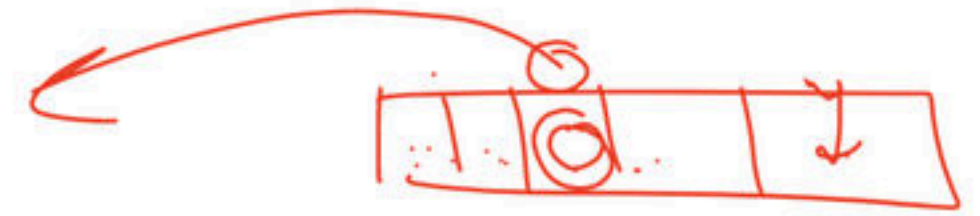
- The TLB is used with page tables in the following way. The TLB contains only a few of the page-table entries. When a logical address is generated by the CPU, its page number is presented to the TLB. If the page number is found, its frame number is immediately available and is used to access memory.
- If the page number is not in the TLB (known as a **TLB Miss**), then a memory reference to the page table must be made.



- Also we add the page number and frame number to the TLB, so that they will be found quickly on the next reference.
- If the TLB is already full of entries, the operating system must select one for replacement i.e. Page replacement policies.
- The percentage of times that a particular page number is found in the TLB is called the **Hit Ratio**.



- **Effective Memory Access Time:**
 - Hit [TLB + Main Memory] + 1-Miss [TLB + 2 Main Memory]
- **TLB** removes the problem of slow access.

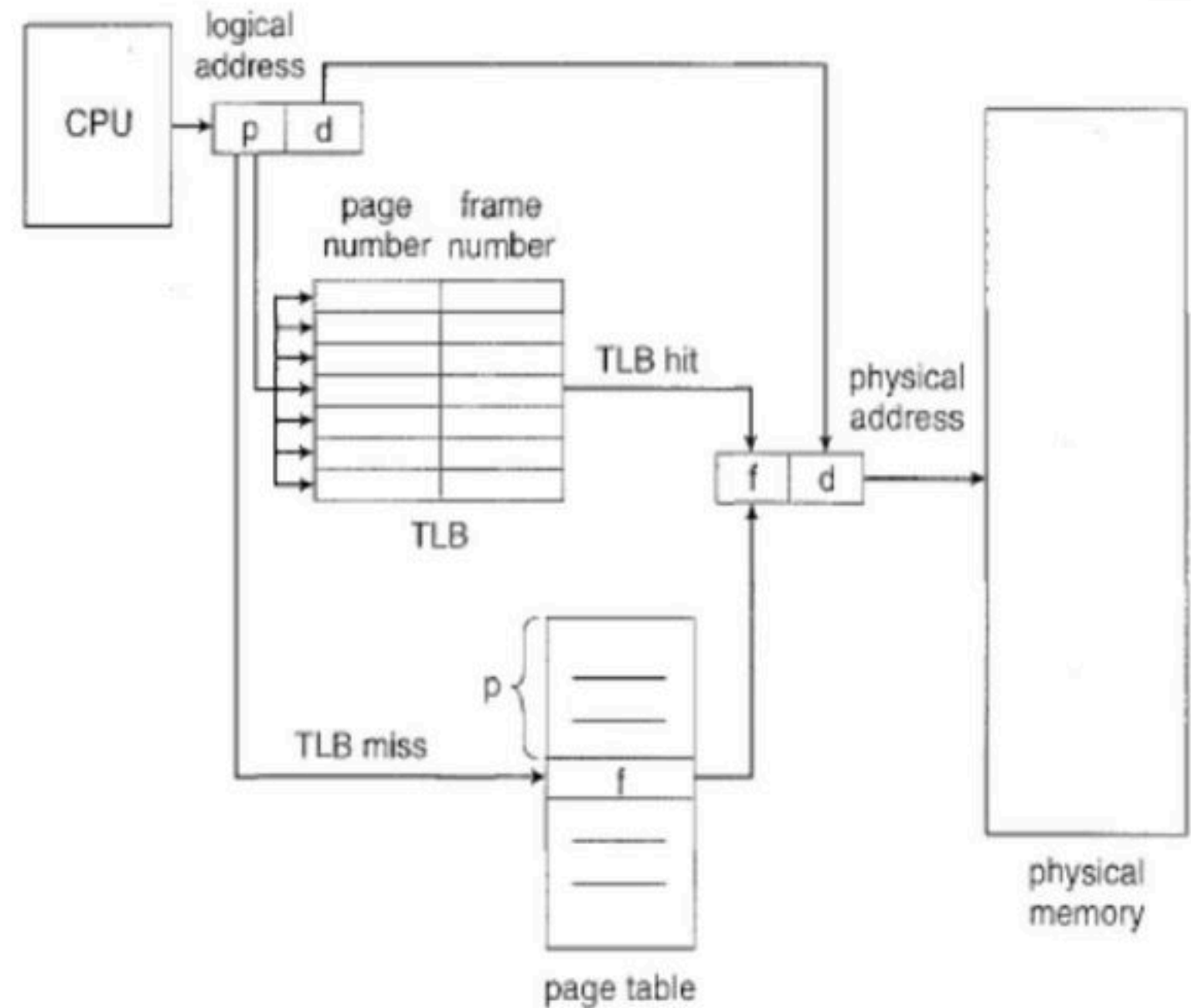


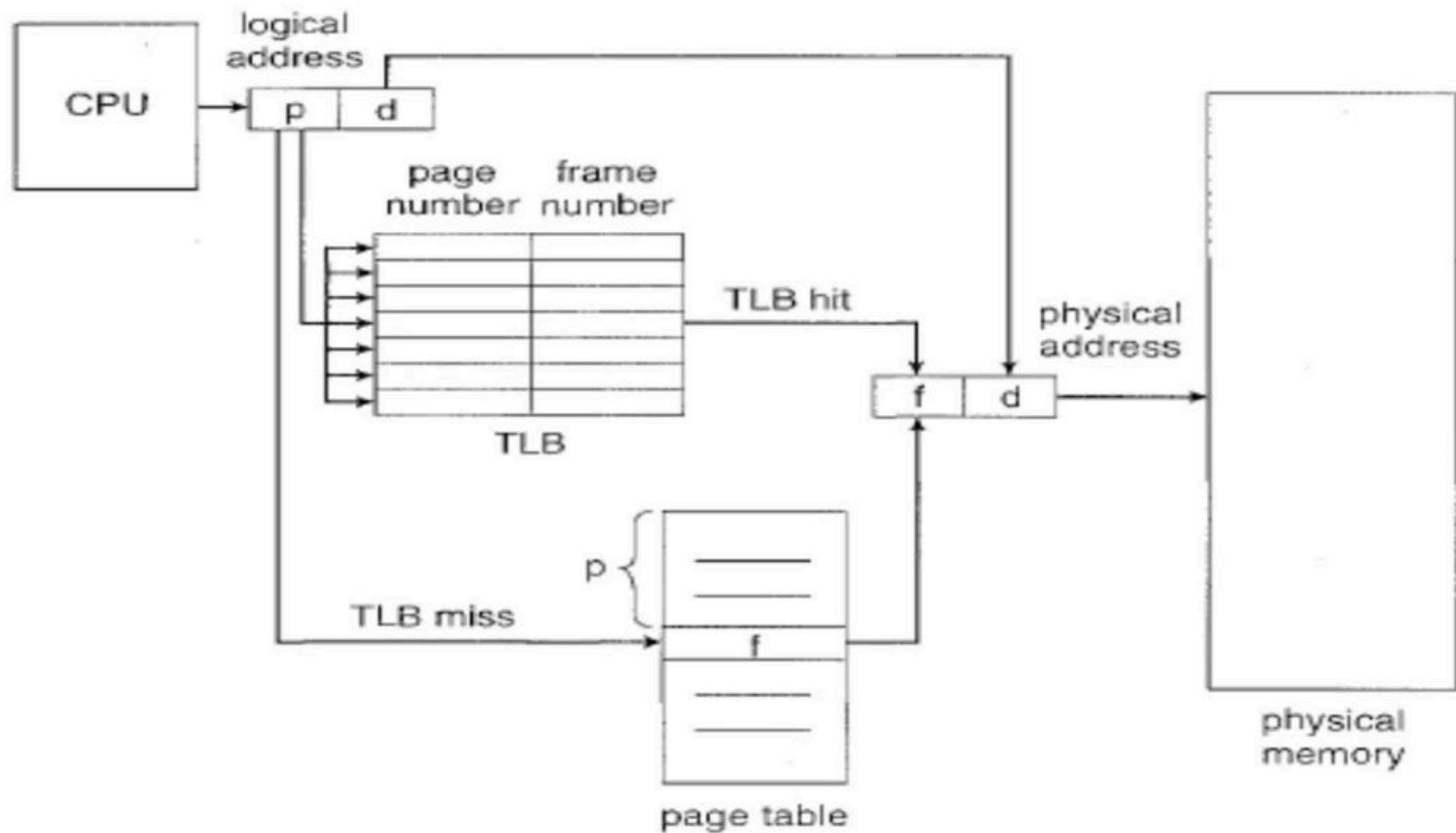
- **Disadvantage of TLB:**

- TLB can hold the data of one process at a time and in case of multiple context switches TLB will be required to flush frequently.

- **Solution:**

- Use multiple TLB's but it will be costly.
- Some TLBs allow certain entries to be **wired down**, meaning that they cannot be removed from the TLB. Typically, TLB entries for kernel code are wired down.





Break

Q Assume that in a certain computer, the virtual addresses are 64 bits long and the physical addresses are 48 bits long. The memory is word addressable. The page size is 8kB and the word size is 4 bytes. The Translation Look-aside Buffer (TLB) in the address translation path has 128 valid entries. At most how many distinct virtual addresses can be translated without any TLB miss? **(GATE-2019) (2 Marks)**

(A) 16×2^{10}

(B) 8×2^{20}

(C) 4×2^{20}

(D) 256×2^{10}

Q In a paging system, it takes 30 ns to search translation Look-a-side Buffer (TLB) and 90 ns to access the main memory. If the TLB hit ratio is 70%, the effective memory access time is : **(NET-JAN-2017)**

a) 48ns

~~b) 147ns~~

c) 120ns

d) 84ns

$$\frac{.7 [30ns + 90ns] + (1-.7) [30ns + 90ns + 90ns]}{\downarrow}$$

147ns

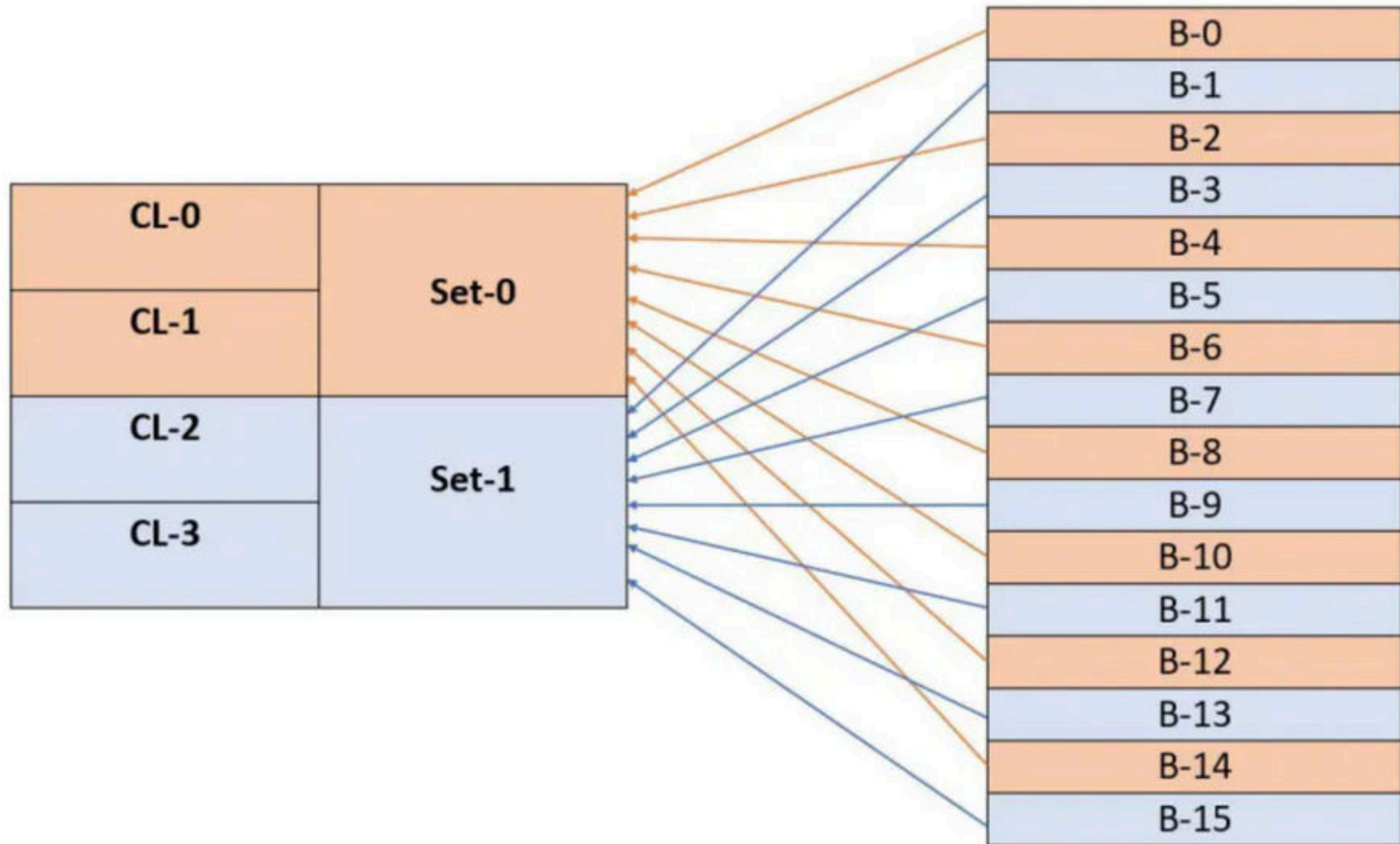
Q A computer system implements a 40-bit virtual address, page size of 8 kilobytes, and a 128-entry translation look-aside buffer (TLB) organized into 32 sets each having four ways. Assume that the TLB tag does not store any process id. The minimum length of the TLB tag in bits is _____ **(GATE-2015) (2 Marks)**

(A) 20

(B) 10

(C) 11

(D) 22



Q Consider a paging hardware with a TLB. Assume that the entire page table and all the pages are in the physical memory. It takes 10 milliseconds to search the TLB and 80 milliseconds to access the physical memory. If the TLB hit ratio is 0.6, the effective memory access time (in milliseconds) is 122. (CS-2014) (2 Marks)

$$= 0.6[10 + 80] + (1 - 0.6)[10 + 80 + 80]$$

122 ms

Q Consider a paging hardware with a TLB. Assume that the entire page table and all the pages are in the physical memory. It takes 10 milliseconds to search the TLB and 80 milliseconds to access the physical memory. If the TLB hit ratio is 0.6, the effective memory access time (in milliseconds) is _____. **(NET-JUNE-2014)**

(A) 120

~~**(B)** 122~~

(C) 124

(D) 118

Q The hit ratio of a Translation Look Aside Buffer (TLAB) is 80%. It takes 20 nanoseconds (ns) to search TLAB and 100 ns to access main memory. The effective memory access time is _____. (**NET-SEP-2013**)

(A) 36 ns

~~**(B)** 140 ns~~

(C) 122 ns

(D) 40 ns

$$.8 [20 + 100] + (1 - .8) [20 + 100 + 100]$$

Q Translation Look-aside Buffer (TLB) is **(NET-DEC-2013)**

(A) a cache-memory in which item to be searched is compared one-by-one with the keys.

(B) a cache-memory in which item to be searched is compared with all the keys simultaneously.

(C) an associative memory in which item to be searched is compared one-by-one with the keys.

(D) an associative memory in which item to be searched is compared with all the keys simultaneously.

Q The virtual address generated by a CPU is 32 bits. The Translation Look-aside Buffer (TLB) can hold total 64-page table entries and a 4-way set associative (i.e. with 4- cache lines in the set). The page size is 4 KB. The minimum size of TLB tag is **(NET-DEC-2013)**

(A) 12 bits

(B) 15 bits

(C) 16 bits

(D) 20 bits

Q A paging system tlb takes 10ns main memory takes 50ns what is effective memory access time if tlb hit ratio is 90%? **(GATE-2008) (1 Marks)**

a) 54
✓

b) 60
✓

~~**c) 65**~~
91

d) 75
3

$$\frac{.9 [10 + 50] + (1 - .9) [10 + 50 + 50]}{2}$$

65ns

Q A CPU generates 32-bit virtual addresses. The page size is 4 KB. The processor has a translation look-aside buffer (TLB) which can hold a total of 128-page table entries and is 4-way set associative. The minimum size of the TLB tag is? **(GATE-2006) (2 Marks)**

(A) 11 bits

(B) 13 bits

(C) 15 bits

(D) 20 bits

Q Which of the following is not a form of memory? (GATE-2002) (1 Marks)

(A) instruction cache

(B) instruction register

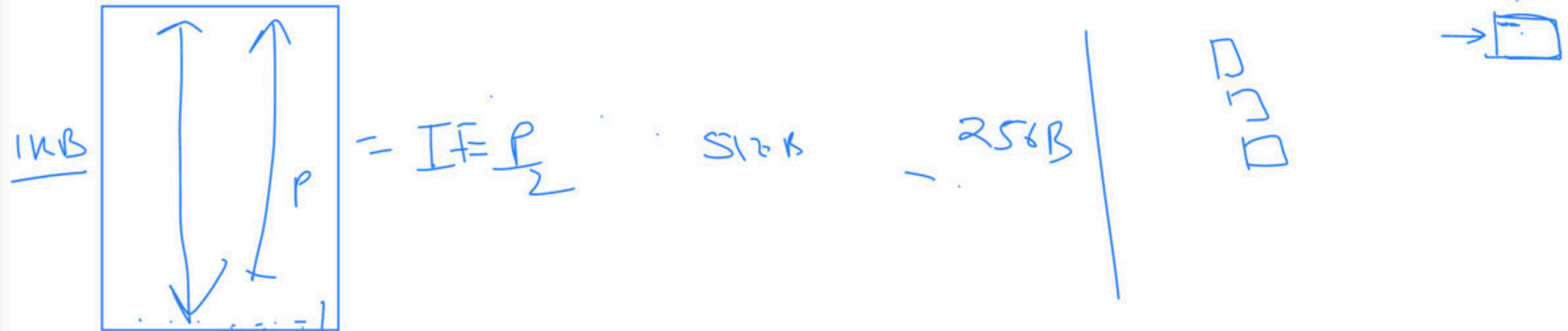
(C) instruction opcode

(D) translation lookaside buffer

Break

Size of Page

- If we increase the size of page ~~table~~ then internal fragmentation increase but size of page table decreases.
- If we decrease the size of page then internal fragmentation decrease but size of page table increases.
- So we have to find what should be the size of the page, where both cost are minimal.

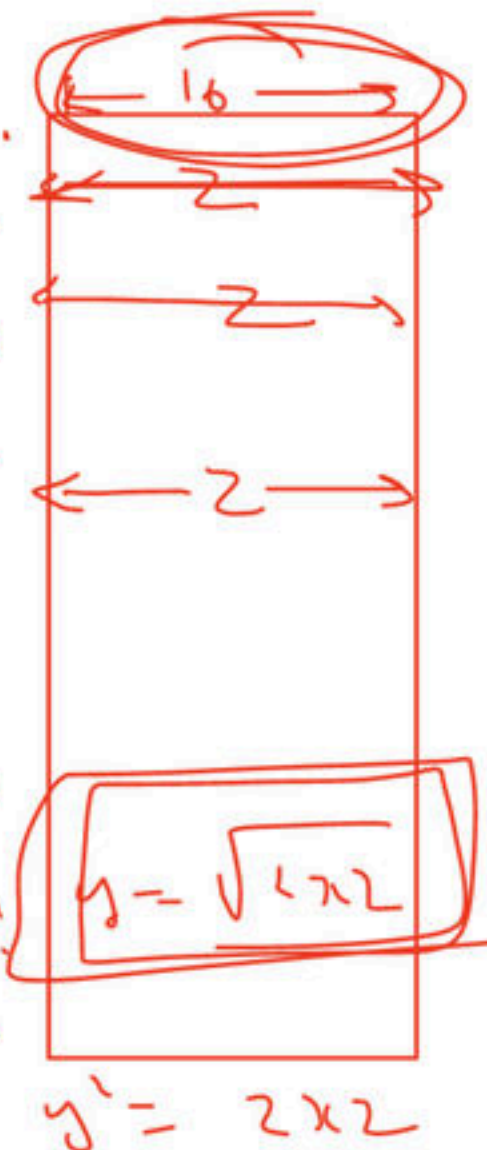


Q For the implementation of a paging scheme, suppose the average process size be x bytes, the page size be y bytes, and each page entry requires z bytes. The optimum page size that minimizes the total overhead due to the page table and the internal fragmentation loss is given by (NET-DEC-2014)

- A) $x/2$ 18 $y \leq z$ b) $xz/2$ 47 ☒ c) $\sqrt{2xz}$ 22 d) $\sqrt{xz}/2$ 13

Process size = x bytes $\rightarrow$ $\frac{x}{y}$

$$P.T = \frac{x}{y} \times z = \frac{xz}{y}$$



Page size = y bytes $\rightarrow$ $\frac{y}{z}$ 1024/13

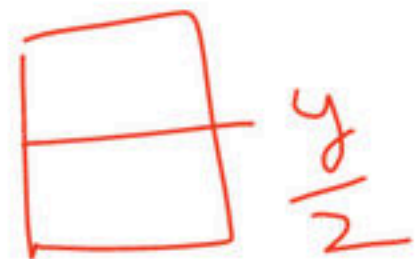
$$\frac{1}{y} \left(\frac{y}{z} \right) + \frac{1}{x} \left(\frac{xz}{y} \right) = 0$$

$$\frac{1 \cdot y^0}{z} + xz(-1)y^{-2} = 0$$

$$\frac{1}{z} - \frac{xz}{y^2} = 0 \quad \frac{1}{z} = \frac{xz}{y^2}$$

$$y^2 =$$

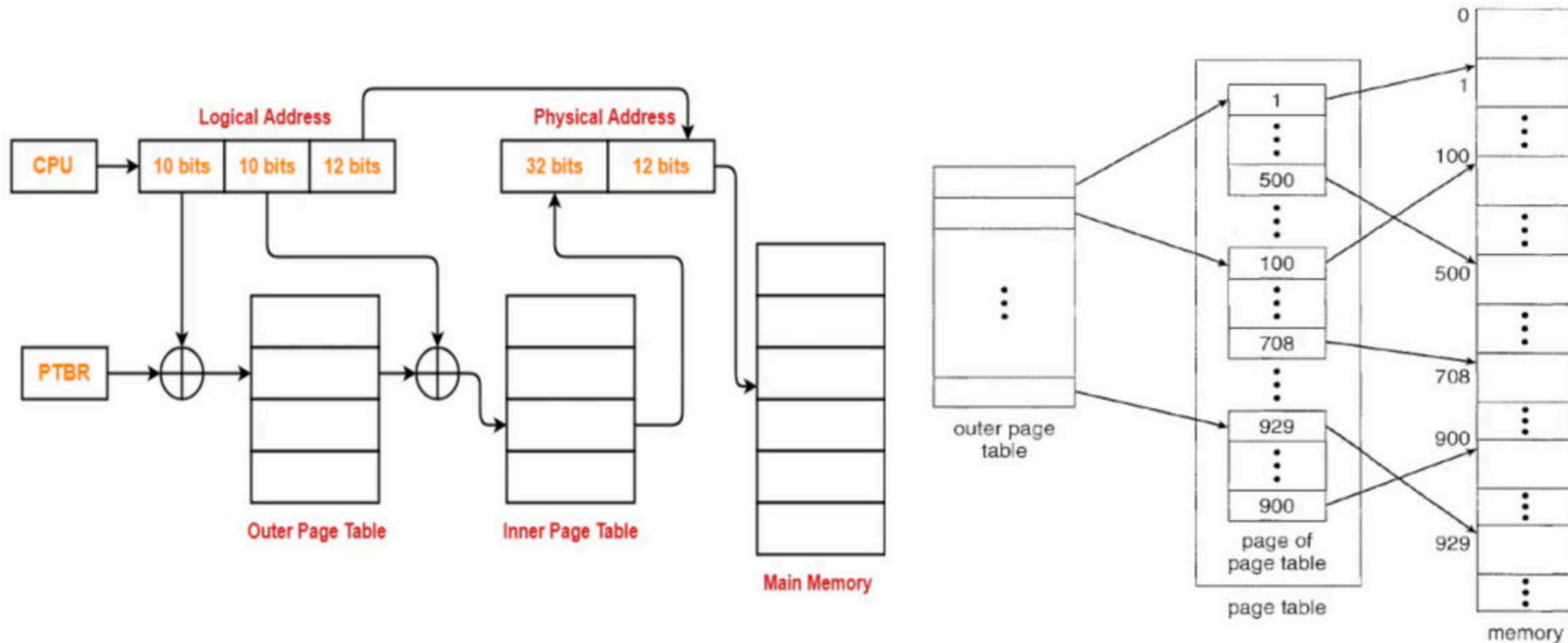
$$n \cdot x^{n-1}$$



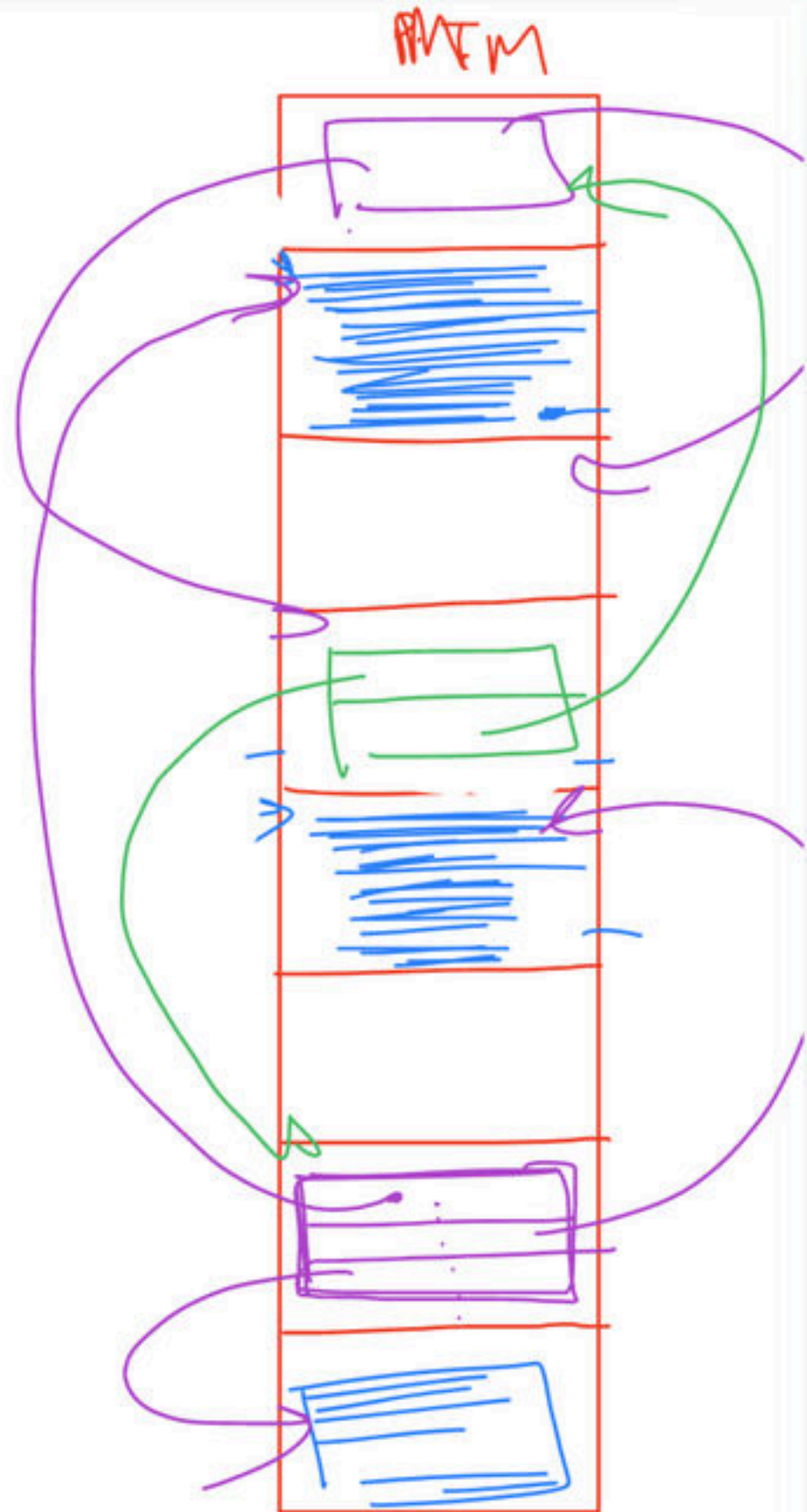
$$y = \sqrt{2xz}$$

Multilevel Paging / Hierarchical Paging

- Modern systems support a large logical address space (2^{32} to 2^{64}).
- In such cases, the page table itself becomes excessively large and can contain millions of entries and can take a lot of space in memory, so cannot be accommodated into a single frame.
- A simple solution to this is to divide page table into smaller pieces.
- One way is to use a **two-level paging algorithm**, in which the page table itself is also paged.



$$\frac{2MB}{1MB} = \frac{2048B}{1K} \textcircled{2}$$



Ch-65

Per-4

I-32

1-100

1-100

1-100

P₁

P₂

P₃

V-6

Ch-5

Per-4

I-32

1-10

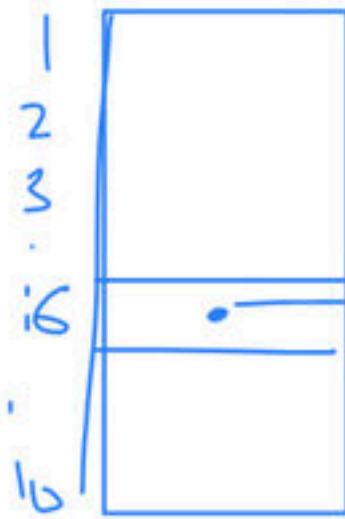
1-10

1-10

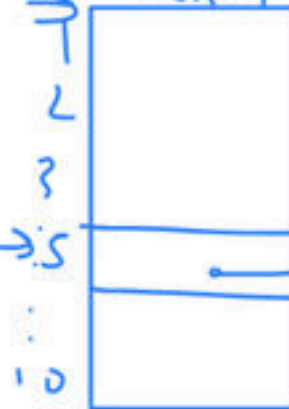
1-100

65,432

V.T



Ch.T



P.T

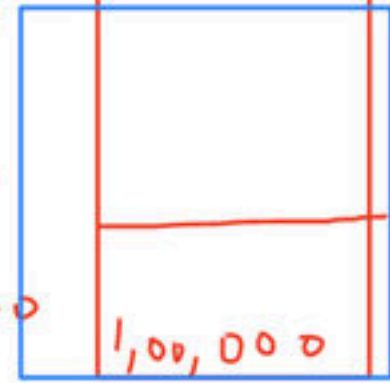
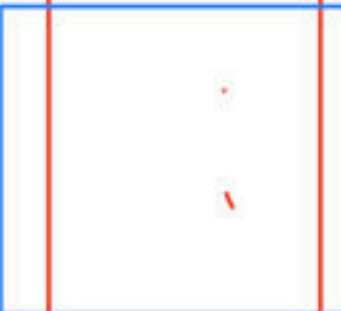
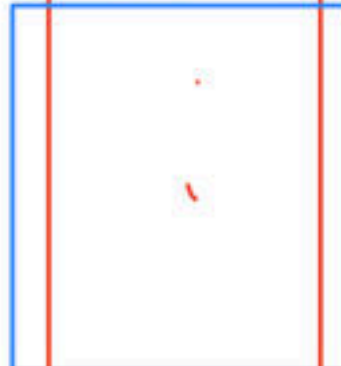
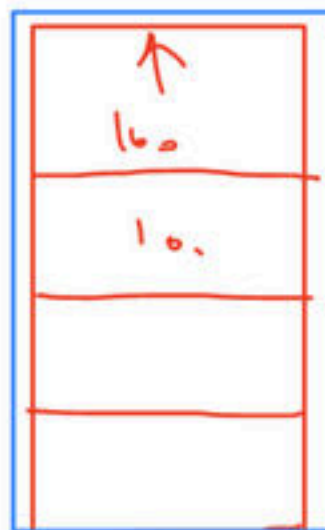


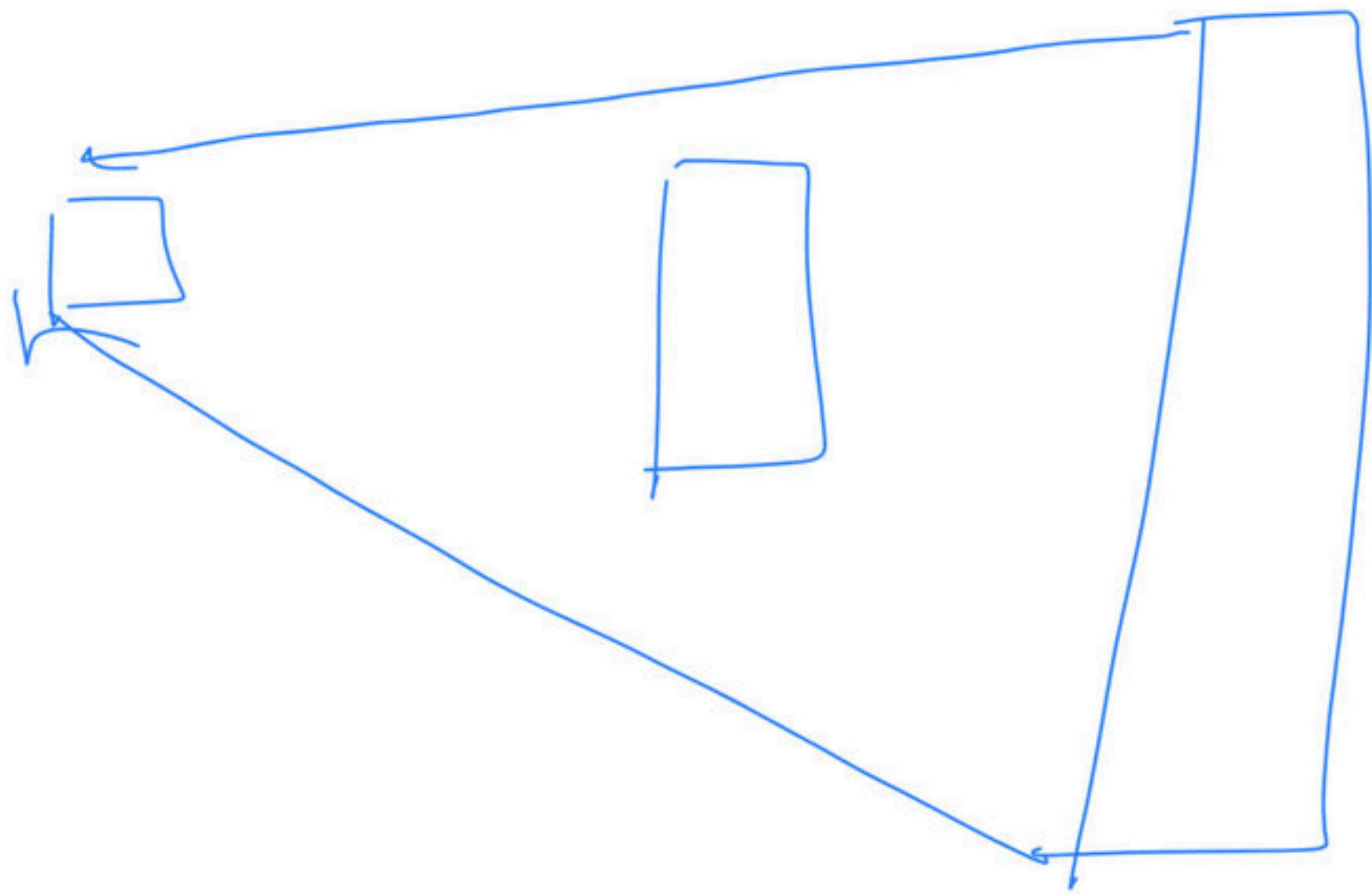
P.A

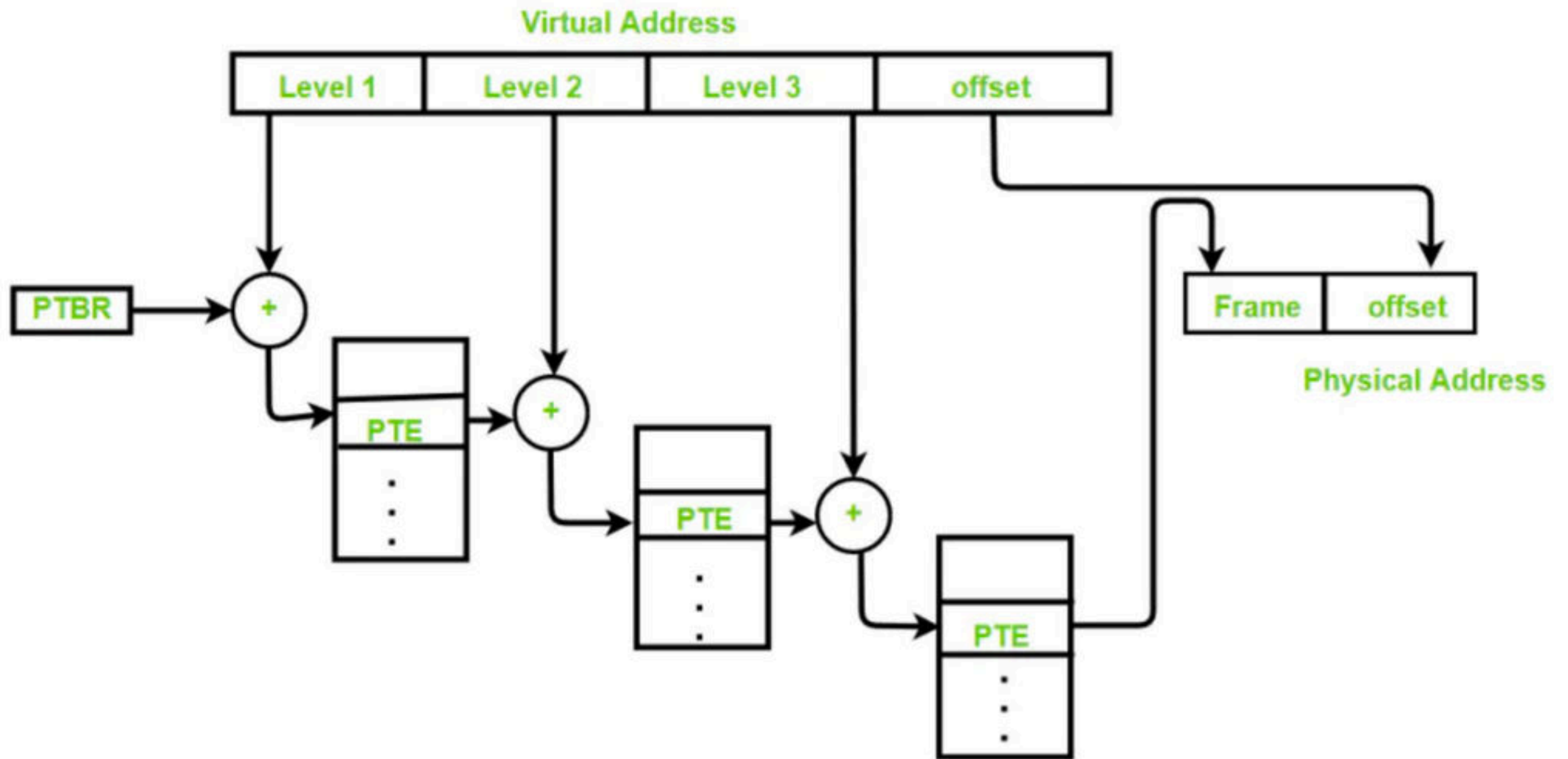
100

1000

1,00,000







3 Level paging system

Q A multilevel page table is preferred in comparison to a single level page table for translating virtual address to physical address because **(GATE-2009) (1 Marks)**

(A) It reduces the memory access time to read or write a memory location.

(B) It helps to reduce the size of page table needed to implement the virtual address space of a process.

(C) It is required by the translation lookaside buffer.

(D) It helps to reduce the number of page faults in page replacement algorithms.

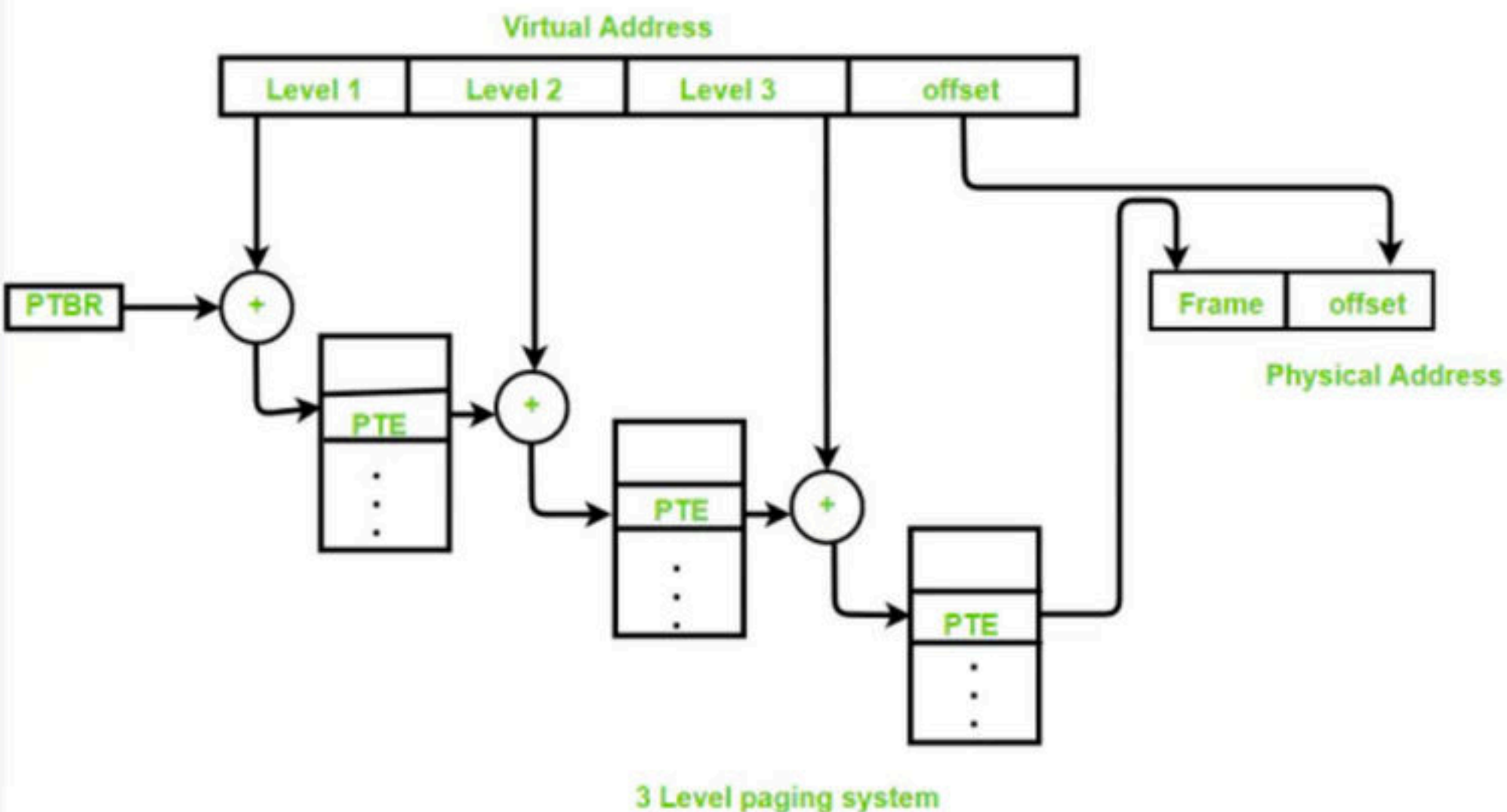
Q A processor uses 36-bit physical addresses and 32-bit virtual addresses, with a page frame size of 4 Kbytes. Each page table entry is of size 4 bytes. A three level page table is used for virtual to physical address translation, where the virtual address is used as follows • Bits 30-31 are used to index into the first level page table • Bits 21-29 are used to index into the second level page table • Bits 12-20 are used to index into the third level page table, and • Bits 0-11 are used as offset within the page The number of bits required for addressing the next level page table (or page frame) in the page table entry of the first, second and third level page tables are respectively **(GATE-2008) (2 Marks)**

(A) 20, 20 and 20

(B) 24, 24 and 24

(C) 24, 24 and 20

(D) 25, 25 and 24



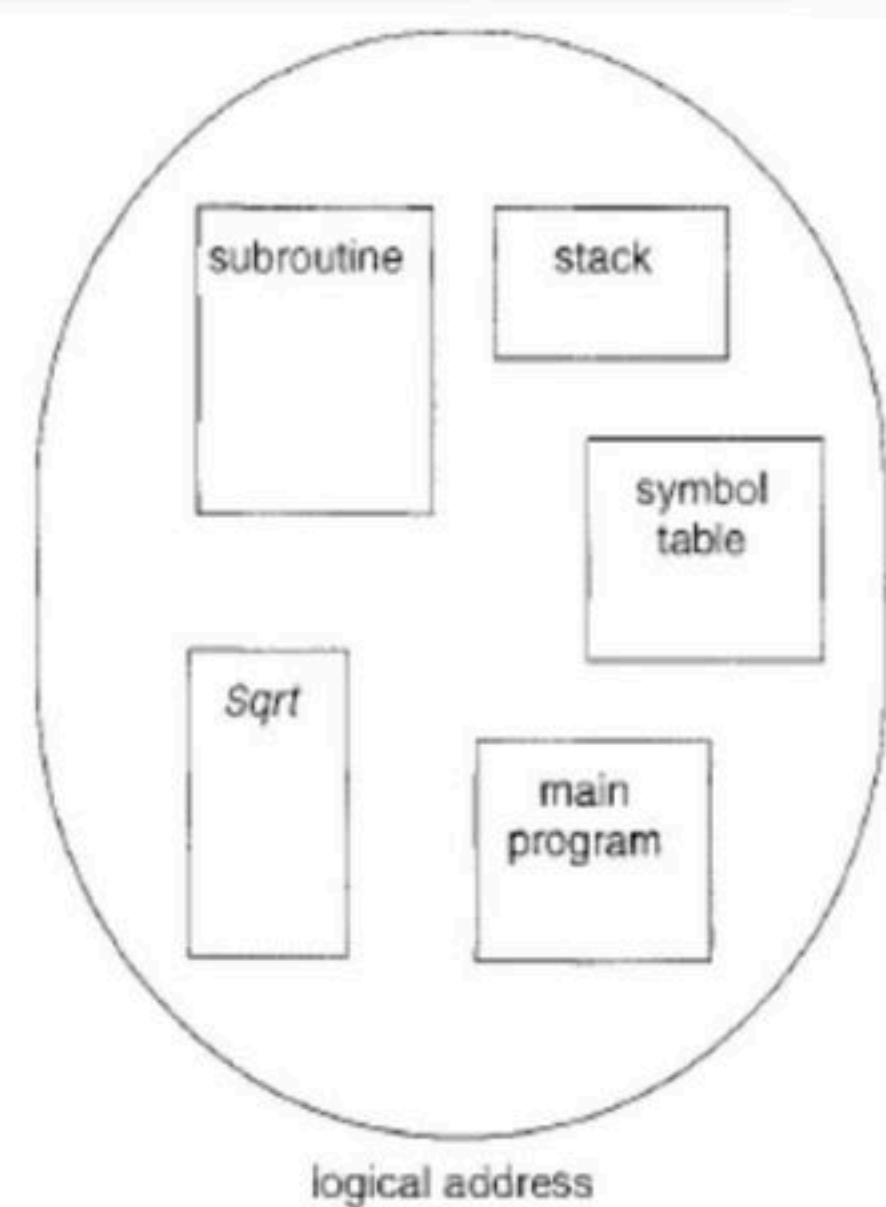
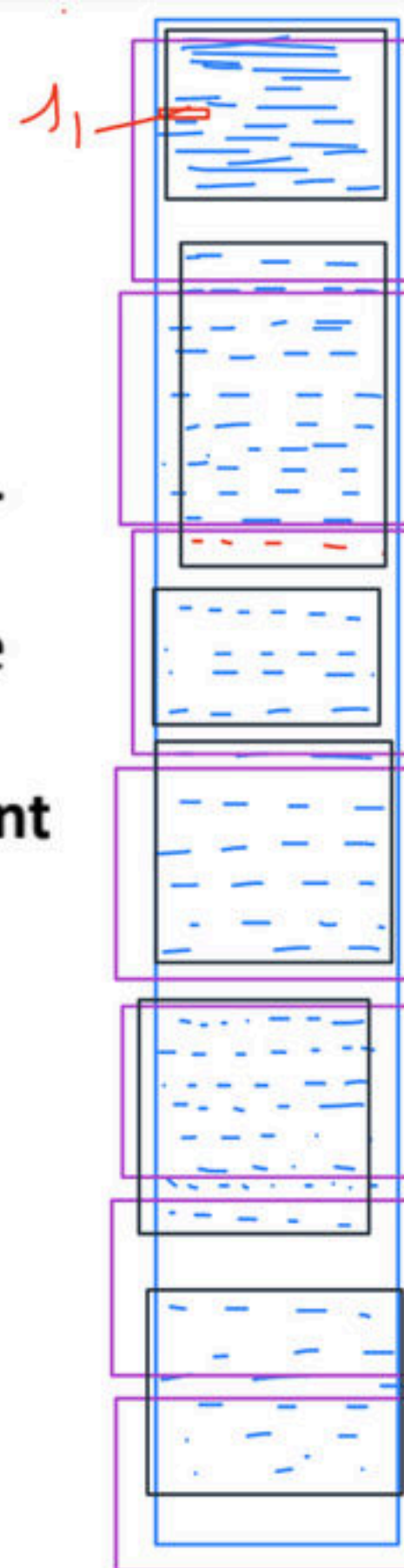
Q In a system with 32 bit virtual addresses and 1 KB page size, use of one-level page tables for virtual to physical address translation is not practical because of **(GATE-2003) (1 Marks)**

- (A)** the large amount of internal fragmentation
- (B)** the large amount of external fragmentation
- (C)** the large memory overhead in maintaining page tables
- (D)** the large computation overhead in the translation process

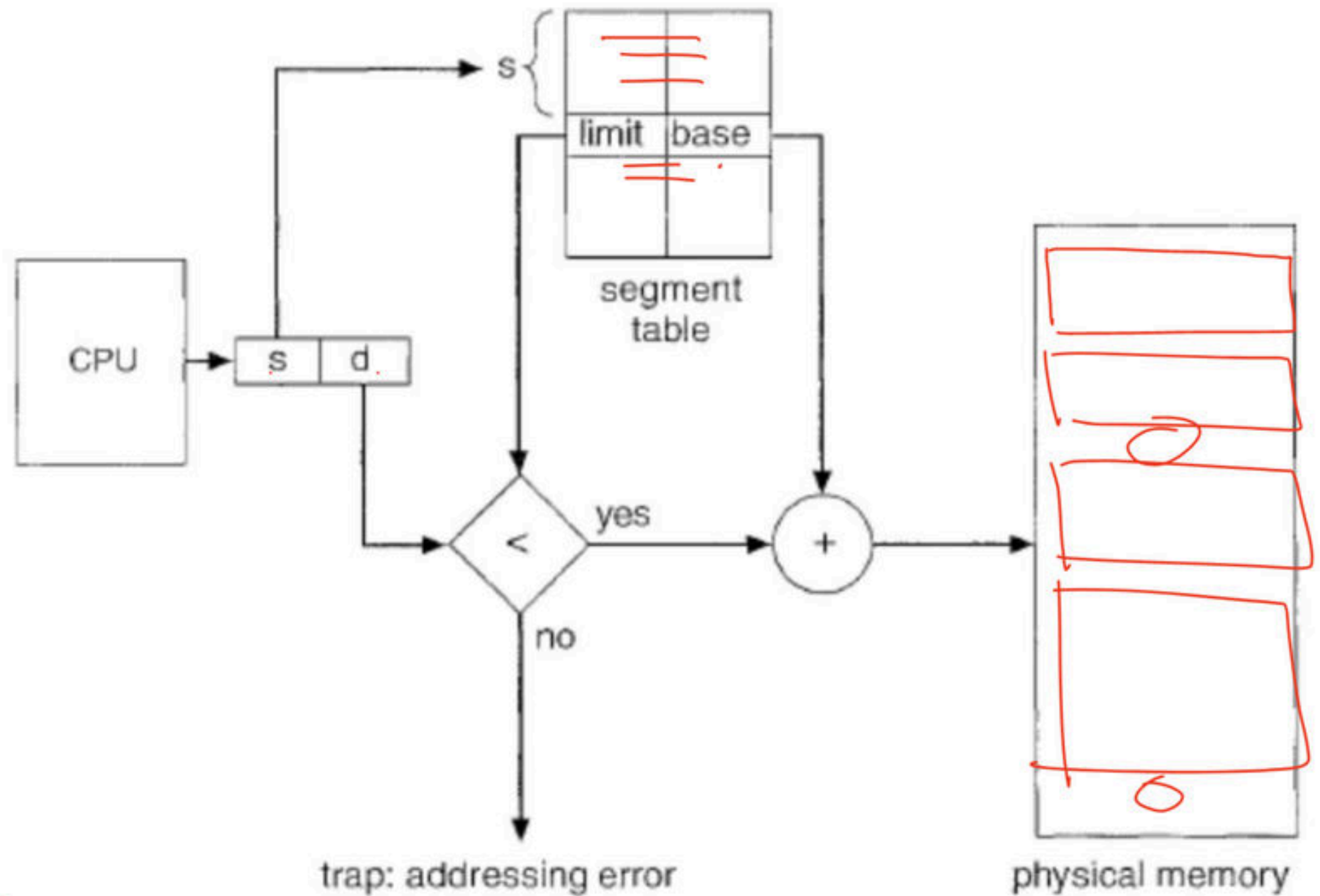
Break

Segmentation

- Paging is unable to separate the user's view of memory from the actual physical memory.
- Segmentation is a memory-management scheme that supports this user view of memory.
- A logical address space is a collection of segments.
- Each segment has a name and a length. The addresses specify both the segment name and the offset within the segment. The user therefore specifies each address by two quantities: **a segment name and an offset**.
- Thus, a logical address consists of a two tuple:
- **<segment-number, offset>**.
- Segments can be of variable lengths unlike pages and are stored in main memory.

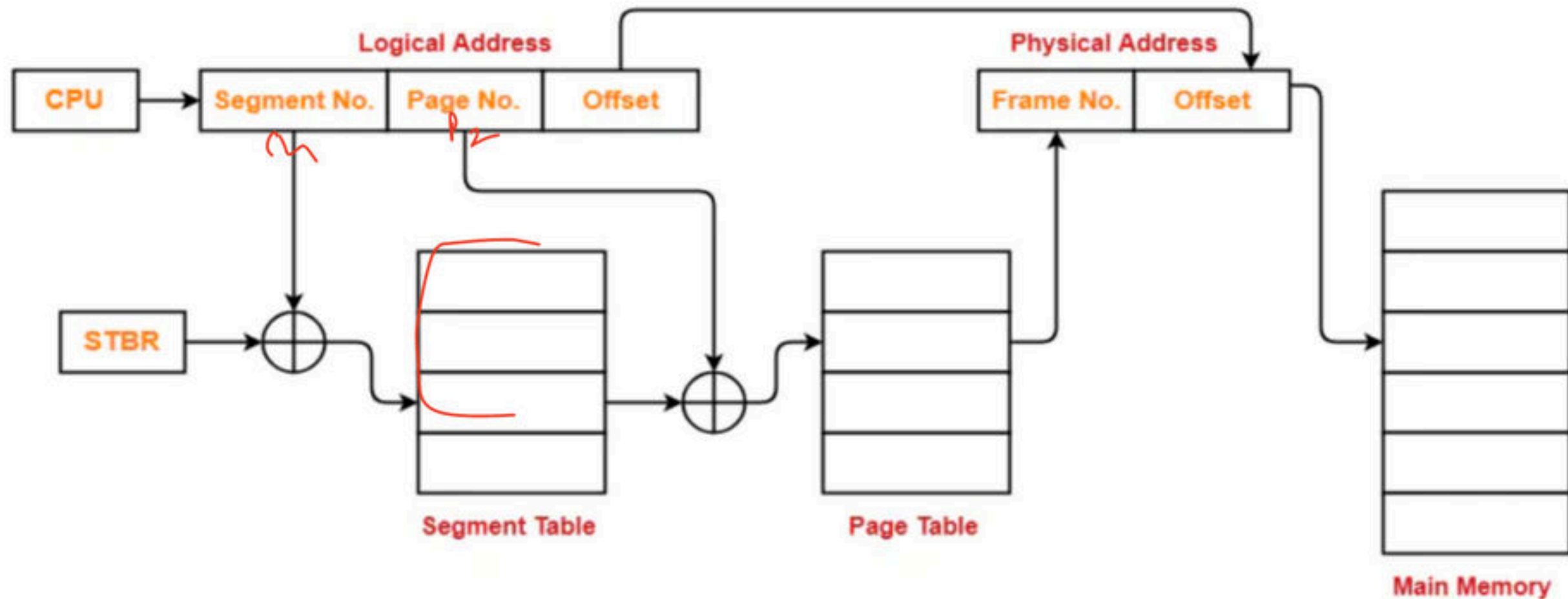


- **Segment Table:** Each entry in the segment table has a **segment base** and a **segment limit**. The segment base contains the starting physical address where the segment resides in memory, and the segment limit specifies the length of the segment.
- The segment number is used as an index to the segment table.
- The offset d of the logical address must be between 0 and the segment limit. If it is not, we trap to the operating system.
- When an offset is legal, it is added to the segment base to produce the address in physical memory of the desired byte.
- Segmentation suffers from **External Fragmentation**.



Segmentation with Paging

- Since segmentation also suffers from external fragmentation, **it is better to divide the segments into pages as the segments size increases.**
- In segmentation with paging scheme a process is divided into segments and further the segments are divided into pages.
- One can argue it is segmentation with paging is quite similar to multilevel paging, but actually it is better, because here when page table is divided the size of partition can be different (as actually the size of different chapters can be different).



Translating Logical Address into Physical Address

- All properties of segmentation with paging is same as multilevel paging

Q A computer uses 46-bit virtual address, 32-bit physical address, and a three-level paged page table organization. The page table base register stores the base address of the first-level table (T1), which occupies exactly one page. Each entry of T1 stores the base address of a page of the second-level table (T2). Each entry of T2 stores the base address of a page of the third-level table (T3). Each entry of T3 stores a page table entry (PTE). The PTE is 32 bits in size. The processor used in the computer has a 1 MB 16 way set associative virtually indexed physically tagged cache. The cache block size is 64 bytes.

What is the size of a page in KB in this computer? **(GATE-2013) (2 Marks)**

(A) 2

(B) 4

(C) 8

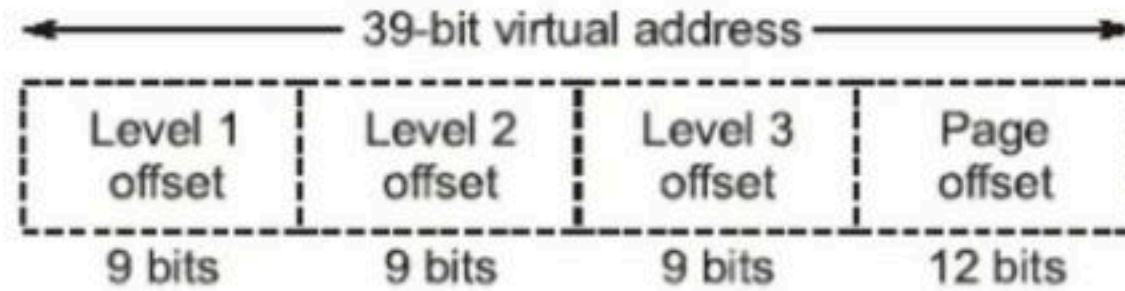
(D) 16

Q.21 In the context of operating systems, which of the following statements is/are correct with **(Gate-2021) (1 Marks)** respect to paging?

- (a) Paging incurs memory overheads.
- (b) Multi-level paging is necessary to support pages of different sizes.
- (c) Page size has no impact on internal fragmentation.
- (d) Paging helps solve the issue of external fragmentation.

Q.52 Consider a three-level page table to translate a 39-bit virtual address to a physical address as shown below:

(GATE- 2021)



The page size is 4 KB ($1 \text{ KB} = 2^{10}$ bytes) and page table entry size at every level is 8 bytes. A process P is currently using 2 GB ($1 \text{ GB} = 2^{30}$ bytes) virtual memory which is mapped to 2 GB of physical memory. The minimum amount of memory required for the page table of P across all levels is _____ KB.